Low-Rank Linear Maps in Machine Learning: From the SVD to LoRA

Arhaan Aggarwal

August 2026

Abstract

This project studies the problem of approximating a matrix by one of rank at most r and asks how such an approximation affects the behavior of a trained neural network. The theoretical component develops the singular value decomposition and proves the Eckart–Young theorem, which states that the rank-r truncated SVD minimizes Frobenius-norm reconstruction error and that the minimum error is determined exactly by the discarded singular-value tail. The computational component trains a $784 \to 256 \to 128 \to 10$ multilayer perceptron on Fashion–MNIST, analyzes its first two weight matrices, and replaces them with factorized truncated-SVD approximations over a range of ranks without retraining. The directly computed reconstruction errors matched the Eckart–Young formula to within 10^{-13}, while both weight matrices exhibited gradually decaying spectra rather than exact low rank. The main empirical finding is that Frobenius reconstruction quality did not directly determine classification accuracy. In the single deterministic run, using rank 48 for both selected layers reduced the full model from 235,146 to 70,026 parameters, a 70.22% reduction and a 3.36× compression factor, while test accuracy decreased only from 88.38% to 88.02%. A joint rank-32 factorization removed 79.91% of the parameters with a 1.13-percentage-point accuracy loss, and the second hidden layer was substantially more tolerant of aggressive compression than the first. On a rotated Fashion–MNIST adaptation task, directly learned LoRA updates outperformed post hoc SVD approximations of a full-rank update at very small ranks, although the difference largely disappeared at higher ranks. Overall, the results show that truncated SVD is optimal for matrix reconstruction, but useful neural-network ranks are determined by layer- and task-dependent behavior.

1 Introduction

Many of the operations performed by a neural network are large matrix multiplications. A fully connected layer maps an input vector $x \in \mathbb{R}^{d_{\text{in}}}$ to an output vector $y \in \mathbb{R}^{d_{\text{out}}}$ according to

$$y = Wx + b, \tag{1}$$

where $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the layer's weight matrix and b is its bias vector. The entries of W encode the relationships learned by the network during training. However, a dense weight matrix contains

$$d_{\text{in}} d_{\text{out}} \tag{2}$$

parameters and requires a comparable number of arithmetic operations each time the layer is evaluated. Consequently, large weight matrices often account for most of a model's storage requirements and a substantial portion of its computational cost.

In the neural network studied in this project, whose architecture is

$$784 \longrightarrow 256 \longrightarrow 128 \longrightarrow 10, \tag{3}$$

the first weight matrix has size 256×784 and therefore contains

$$256 \cdot 784 = 200{,}704 \tag{4}$$

parameters. The second weight matrix has size 128×256 and contains an additional

$$128 \cdot 256 = 32{,}768 \tag{5}$$

parameters. This makes the two matrices natural targets for compression.

A matrix can nevertheless be large in dimension without using all of its possible transformation directions equally. If much of the action of W is concentrated in a smaller number r of important directions, then W may be approximated by a low-rank factorization

$$W \approx BA, \tag{6}$$

where

$$A \in \mathbb{R}^{r \times d_{\text{in}}} \qquad \text{and} \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}. \tag{7}$$

Instead of storing $d_{\text{in}} d_{\text{out}}$ weights, this representation stores only

$$r\,(d_{\text{in}} + d_{\text{out}}) \tag{8}$$

weights. It therefore produces genuine compression whenever

$$r\,(d_{\text{in}} + d_{\text{out}}) < d_{\text{in}} d_{\text{out}}. \tag{9}$$

The corresponding computation is performed as

$$x \longmapsto Ax \longmapsto B(Ax), \tag{10}$$

with the r-dimensional intermediate vector acting as an information bottleneck. When the discarded directions are weak or redundant, this factorization can substantially reduce the parameter count and potentially the computational cost while preserving much of the original transformation.

The singular value decomposition provides a principled method for constructing such an approximation. If

$$W = U\Sigma V^\top = \sum_{i=1}^{p} \sigma_i u_i v_i^\top, \tag{11}$$

where

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0, \tag{12}$$

then the rank-r truncated singular value decomposition retains only the r terms corresponding to the largest singular values:

$$W_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top. \tag{13}$$

The Eckart–Young theorem states that this is the optimal rank-r approximation under the Frobenius norm:

$$\min_{\operatorname{rank}(C)\leq r} \|W - C\|_F = \|W - W_r\|_F = \sqrt{\sum_{i=r+1}^{p} \sigma_i^2}. \tag{14}$$

Here, the symbol C represents any matrix of rank at most r. Thus, among all matrices that use at most r independent transformation directions, the truncated SVD preserves the greatest possible amount of the original matrix's Frobenius energy.

This theorem, however, answers a question about matrices rather than a question about neural-network performance. It guarantees that W_r is geometrically close to W, but it does not guarantee that replacing W with W_r will preserve the network's predictions. The Frobenius norm treats approximation error according to the size of the discarded matrix directions, whereas classification performance depends on the inputs encountered by the model, the nonlinear activation functions, the later layers of the network, and the importance of each direction for distinguishing among classes.

A small reconstruction error could therefore remove a direction that is important for classification, while a larger reconstruction error might occur primarily in directions that have little effect on the model's decisions. Matrix-level approximation quality and model-level predictive performance are consequently related, but they are not identical.

The central question of this project is therefore:

To what extent does the mathematically optimal rank-r approximation of a neural-network weight matrix preserve the network's classification accuracy, and how does the choice of rank determine the tradeoff among reconstruction error, parameter reduction, and predictive performance?

To investigate this question, we study a fully connected classifier trained on the Fashion-MNIST image-classification dataset. We compute the singular-value spectra of its first two weight matrices and replace them with truncated-SVD factorizations at a range of ranks. We examine three configurations: compressing the first layer alone, compressing the second layer alone, and compressing both layers simultaneously.

The compressed models are evaluated without retraining so that the experiment isolates the immediate effect of replacing the original transformations with their low-rank approximations. For each selected rank, we measure the absolute and relative Frobenius reconstruction errors, retained singular-value energy, factorized parameter count, compression ratio, and test-set classification accuracy.

The project makes both theoretical and computational contributions. The theoretical component develops the required linear-algebraic foundations, provides a geometric interpretation and proof of the Eckart–Young theorem, derives the singular-value formula for the optimal approximation error, and explains precisely when a low-rank factorization reduces the number of parameters.

The theoretical discussion also distinguishes truncated-SVD compression from low-rank adaptation, commonly known as LoRA. Truncated SVD approximates an already trained matrix and is optimal for a matrix-reconstruction objective. In contrast, LoRA freezes a pretrained matrix and learns a low-rank update directly by optimizing a task-dependent training objective. Therefore, truncated SVD compresses the existing transformation, whereas LoRA learns a new low-rank modification to that transformation.

The computational component verifies the Eckart–Young error formula numerically and tests whether matrix-level optimality translates into the preservation of model-level behavior. By placing the theoretical reconstruction guarantees alongside empirical classification-accuracy measurements, the project illustrates both the usefulness and the limitations of low-rank matrix approximation in machine learning.

2 Linear-Algebra Background

The Eckart–Young theorem concerns the approximation of a matrix by another matrix of lower rank. To state and understand the theorem, we first review matrices as linear transformations, rank, orthogonality, the Frobenius norm, and the singular value decomposition. These concepts allow a matrix to be expressed as a sum of mutually orthogonal rank-one components, after which low-rank approximation can be understood as retaining only the most important components.

2.1 Matrices as Linear Transformations

Let

$$A \in \mathbb{R}^{m \times n}.$$

The matrix A defines a linear transformation

$$A : \mathbb{R}^n \longrightarrow \mathbb{R}^m, \qquad x \longmapsto Ax.$$

The space $\mathbb{R}^n$ is the *domain* of the transformation, while $\mathbb{R}^m$ is its *codomain*. The domain contains the possible input vectors, and the codomain contains the possible types of output vectors.

The codomain should be distinguished from the *image* of A. The image is the set of vectors that can actually be produced by the transformation:

$$\operatorname{im}(A) = \{Ax : x \in \mathbb{R}^n\}.$$

The image is a subspace of $\mathbb{R}^m$, but it does not necessarily equal all of $\mathbb{R}^m$.

To see this more concretely, write the columns of A as

$$A = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix},$$

where each $a_j \in \mathbb{R}^m$. For an input vector

$$x = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}^\top,$$

matrix multiplication gives

$$Ax = x_1 a_1 + x_2 a_2 + \cdots + x_n a_n.$$

Therefore,

$$\operatorname{im}(A) = \operatorname{span}\{a_1, \ldots, a_n\}.$$

In other words, every possible output of A is a linear combination of the columns of A.

The *kernel*, or null space, of A is the set of input vectors that are mapped to the zero vector:

$$\ker(A) = \{x \in \mathbb{R}^n : Ax = 0\}.$$

Vectors in the kernel represent input directions that are completely eliminated by the transformation.

The *rank* of A is defined as the dimension of its image:

$$\operatorname{rank}(A) = \dim\big(\operatorname{im}(A)\big).$$

Equivalently, the rank is the maximum number of linearly independent columns of A. It is also equal to the maximum number of linearly independent rows of A.

The rank–nullity theorem states that

$$\operatorname{rank}(A) + \dim\big(\ker(A)\big) = n.$$

Thus, the dimensions of the input space are divided between directions that contribute to the output and directions that are removed by the transformation.

Rank measures the number of independent output directions that a matrix can produce. If

$$\operatorname{rank}(A) = r,$$

then every output Ax lies in an r-dimensional subspace of $\mathbb{R}^m$. In particular, when

$$r \ll \min(m, n),$$

the matrix is said to be *low rank*. Although the matrix may contain mn entries, its action is restricted to a much smaller number of independent directions.

This interpretation is especially relevant for neural networks. Ignoring the bias term, a fully connected layer maps an input x to Ax. If A is low rank, then the layer's outputs are restricted to a lower-dimensional subspace, allowing the transformation to be represented using fewer parameters.

2.2 Orthogonality and Norms

For vectors $x, y \in \mathbb{R}^n$, their Euclidean inner product is

$$\langle x, y \rangle = x^\top y = \sum_{i=1}^{n} x_i y_i.$$

The associated Euclidean norm is

$$\|x\|_2 = \sqrt{\langle x, x \rangle} = \sqrt{\sum_{i=1}^{n} x_i^2}.$$

The Euclidean norm measures the usual length of a vector.

Two vectors x and y are *orthogonal* if

$$\langle x, y \rangle = 0.$$

Geometrically, nonzero orthogonal vectors point in perpendicular directions.

A collection of vectors

$$q_1, \ldots, q_k$$

is *orthonormal* if

$$q_i^\top q_j = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Equivalently,

$$q_i^\top q_j = \delta_{ij},$$

where δ_{ij} is the Kronecker delta. An orthonormal basis is a basis consisting of mutually orthogonal unit vectors.

If a matrix

$$Q = \begin{bmatrix} q_1 & \cdots & q_k \end{bmatrix}$$

has orthonormal columns, then

$$Q^\top Q = I_k.$$

When Q is square, it is called an *orthogonal matrix*. In this case,

$$Q^\top Q = QQ^\top = I$$

and

$$Q^{-1} = Q^\top.$$

Orthogonal matrices preserve Euclidean inner products and lengths:

$$\langle Qx, Qy\rangle = \langle x, y\rangle$$

and

$$\|Qx\|_2 = \|x\|_2.$$

They therefore represent rotations, reflections, or combinations of these operations without stretching the underlying space.

For matrices $A, B \in \mathbb{R}^{m\times n}$, the *Frobenius inner product* is defined by

$$\langle A, B\rangle_F = \operatorname{tr}(A^\top B) = \sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}B_{ij},$$

where $\operatorname{tr}(\cdot)$ denotes the trace of a square matrix. The corresponding *Frobenius norm* is

$$\|A\|_F = \sqrt{\langle A, A\rangle_F} = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2}.$$

Thus, the Frobenius norm treats the entries of a matrix as the coordinates of a vector and measures their total Euclidean magnitude.

The Frobenius norm is useful for matrix approximation because

$$\|A - B\|_F$$

measures the total entrywise difference between two matrices A and B. A smaller Frobenius distance indicates that the entries of B more closely approximate the corresponding entries of A.

2.3 Singular Value Decomposition

Every matrix $A \in \mathbb{R}^{m\times n}$ has a *singular value decomposition*, or SVD, of the form

$$A = U\Sigma V^\top,$$

where

$$U \in \mathbb{R}^{m\times m}, \qquad V \in \mathbb{R}^{n\times n}$$

are orthogonal matrices, and

$$\Sigma \in \mathbb{R}^{m \times n}$$

is a rectangular diagonal matrix.

Let

$$p = \min(m, n).$$

The diagonal entries of Σ are the singular values of A, written in nonincreasing order:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0.$$

Thus, Σ has the form

$$\Sigma = \begin{bmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \ddots & \\ 0 & & & \sigma_p \end{bmatrix},$$

with any additional entries equal to zero.

Write the columns of U and V as

$$U = \begin{bmatrix} u_1 & \cdots & u_m \end{bmatrix}$$

and

$$V = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}.$$

The vectors u_i are called the *left singular vectors*, while the vectors v_i are called the *right singular vectors*. Since U and V are orthogonal,

$$u_i^\top u_j = \delta_{ij} \qquad \text{and} \qquad v_i^\top v_j = \delta_{ij}.$$

For each $i \leq p$, the SVD satisfies

$$Av_i = \sigma_i u_i.$$

This equation provides a geometric interpretation of the decomposition. The matrix $V^\top$ first expresses an input vector in the orthonormal basis of right singular vectors. The matrix Σ then scales the i-th direction by σ_i, and U maps the resulting coordinates into the corresponding left singular-vector directions.

The singular values therefore measure how strongly the matrix acts along particular orthogonal input directions. Larger singular values correspond to directions that are amplified more strongly, while a zero singular value corresponds to an input direction that is eliminated.

The number of positive singular values is equal to the rank of A:

$$\operatorname{rank}(A) = \#\{i : \sigma_i > 0\}.$$

2.4 Rank-One Decomposition

The SVD can be expanded as a sum of outer products:

$$A = \sum_{i=1}^{p} \sigma_i u_i v_i^\top.$$

For any vector $x \in \mathbb{R}^n$, the action of the i-th term is

$$\left(\sigma_i u_i v_i^\top\right) x = \sigma_i \left(v_i^\top x\right) u_i.$$

The scalar $v_i^\top x$ measures the component of x in the direction v_i. The transformation then scales this component by σ_i and sends it in the output direction u_i.

The outer product

$$u_i v_i^\top$$

has image

$$\operatorname{im}\left(u_i v_i^\top\right) = \operatorname{span}\{u_i\}.$$

Consequently, it has rank one. More precisely, the term

$$\sigma_i u_i v_i^\top$$

has rank one whenever $\sigma_i > 0$; if $\sigma_i = 0$, it is the zero matrix and has rank zero.

The matrices $u_i v_i^\top$ are mutually orthonormal under the Frobenius inner product. Indeed,

$$\begin{aligned}
\left\langle u_i v_i^\top, u_j v_j^\top \right\rangle_F &= \operatorname{tr}\left(\left(u_i v_i^\top\right)^\top u_j v_j^\top\right) \\
&= \operatorname{tr}\left(v_i u_i^\top u_j v_j^\top\right) \\
&= \left(u_i^\top u_j\right)\left(v_i^\top v_j\right) \\
&= \delta_{ij}.
\end{aligned}$$

Therefore, the rank-one components in the SVD point in orthogonal directions when matrices are viewed as elements of the Frobenius inner-product space.

Using this orthogonality, the squared Frobenius norm of A is

$$\begin{aligned}
\|A\|_F^2 &= \left\langle \sum_{i=1}^{p} \sigma_i u_i v_i^\top, \sum_{j=1}^{p} \sigma_j u_j v_j^\top \right\rangle_F \\
&= \sum_{i=1}^{p} \sum_{j=1}^{p} \sigma_i \sigma_j \left\langle u_i v_i^\top, u_j v_j^\top \right\rangle_F \\
&= \sum_{i=1}^{p} \sigma_i^2.
\end{aligned}$$

Hence,

$$\boxed{\|A\|_F^2 = \sum_{i=1}^{p} \sigma_i^2}.$$

This identity shows that the squared singular values describe how the total Frobenius magnitude, or "energy," of the matrix is distributed among its orthogonal rank-one components.

2.5 Truncated Singular Value Decomposition

For an integer r satisfying

$$0 \le r \le p,$$

the rank-r truncated SVD of A is defined by retaining only the first r singular components:

$$A_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top.$$

Because each term maps into the one-dimensional space spanned by u_i, every output of A_r lies in

$$\operatorname{span}\{u_1, \dots, u_r\}.$$

Therefore,

$$\operatorname{im}(A_r) \subseteq \operatorname{span}\{u_1, \dots, u_r\},$$

which implies

$$\boxed{\operatorname{rank}(A_r) \le r}.$$

If $\sigma_r > 0$, then the first r singular components are all nonzero and

$$\operatorname{rank}(A_r) = r.$$

Subtracting A_r from the full SVD gives

$$\begin{aligned} A - A_r &= \sum_{i=1}^{p} \sigma_i u_i v_i^\top - \sum_{i=1}^{r} \sigma_i u_i v_i^\top \\ &= \sum_{i=r+1}^{p} \sigma_i u_i v_i^\top. \end{aligned}$$

Thus, the approximation error consists exactly of the singular components that were discarded.

Since these components are orthonormal under the Frobenius inner product,

$$\begin{aligned} \|A - A_r\|_F^2 &= \left\| \sum_{i=r+1}^{p} \sigma_i u_i v_i^\top \right\|_F^2 \\ &= \sum_{i=r+1}^{p} \sigma_i^2. \end{aligned}$$

Therefore,

$$\boxed{\|A - A_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2}$$

or, equivalently,

$$\boxed{\|A - A_r\|_F = \sqrt{\sum_{i=r+1}^{p} \sigma_i^2}}.$$

Because the singular values are ordered from largest to smallest, A_r retains the r rank-one components with the greatest Frobenius energy and discards the remaining components. This calculation establishes the error achieved by truncated SVD. The Eckart–Young theorem will show the stronger result that no other matrix of rank at most r can produce a smaller approximation error in the Frobenius norm.

3 Truncated SVD and the Eckart–Young Theorem

The singular value decomposition does more than express a matrix as a sum of rank-one matrices. It also orders those rank-one components according to their singular values, thereby providing a natural method for constructing low-rank approximations.

Let

$$A \in \mathbb{R}^{m \times n}, \qquad p = \min\{m, n\},$$

and write the singular value decomposition of A as

$$A = U\Sigma V^\top = \sum_{i=1}^{p} \sigma_i u_i v_i^\top, \tag{15}$$

where

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0$$

are the singular values of A. The vectors

$$u_1, \ldots, u_p \in \mathbb{R}^m$$

are orthonormal left singular vectors, while

$$v_1, \ldots, v_p \in \mathbb{R}^n$$

are orthonormal right singular vectors.

For an integer r satisfying

$$0 \leq r \leq p,$$

the rank-r truncated singular value decomposition of A is defined by

$$A_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top. \tag{16}$$

By convention, $A_0 = 0$.

Equivalently, if

$$U_r = \begin{bmatrix} u_1 & \cdots & u_r \end{bmatrix}, \qquad V_r = \begin{bmatrix} v_1 & \cdots & v_r \end{bmatrix},$$

and

$$\Sigma_r = \operatorname{diag}(\sigma_1, \ldots, \sigma_r),$$

then

$$A_r = U_r \Sigma_r V_r^\top. \tag{17}$$

Since A_r is a sum of at most r rank-one matrices,

$$\operatorname{rank}(A_r) \le r.$$

The following theorem shows that A_r is not merely one possible rank-r approximation. It is an optimal rank-r approximation in the Frobenius norm.

Theorem 3.1 (Eckart–Young theorem)**.** *Let $A \in \mathbb{R}^{m \times n}$ have singular values*

$$\sigma_1 \ge \sigma_2 \ge \cdots \ge \sigma_p \ge 0, \qquad p = \min\{m, n\}.$$

For $0 \le r \le p$, define

$$A_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top.$$

Then

$$\min_{\operatorname{rank}(B) \le r} \|A - B\|_F = \|A - A_r\|_F = \left(\sum_{i=r+1}^{p} \sigma_i^2 \right)^{1/2}. \tag{18}$$

Equivalently,

$$\min_{\operatorname{rank}(B) \le r} \|A - B\|_F^2 = \|A - A_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2. \tag{19}$$

The theorem was introduced by Eckart and Young [2]. Modern treatments of the singular value decomposition and low-rank approximation can also be found in [3, 7].

Proof. The proof has two main parts. First, we compute the error achieved by A_r. Second, we show that no matrix of rank at most r can achieve a smaller error.

Step 1: Compute the error achieved by A_r. For any i and j, the Frobenius inner product between the rank-one matrices $u_i v_i^\top$ and $u_j v_j^\top$ is

$$\begin{aligned}\left\langle u_i v_i^\top, u_j v_j^\top \right\rangle_F &= \operatorname{tr}\left((u_i v_i^\top)^\top (u_j v_j^\top)\right) && (20)\\ &= \operatorname{tr}\left(v_i u_i^\top u_j v_j^\top\right) && (21)\\ &= (u_i^\top u_j)(v_i^\top v_j). && (22)\end{aligned}$$

Because the left and right singular vectors are orthonormal,

$$u_i^\top u_j = \delta_{ij}, \qquad v_i^\top v_j = \delta_{ij},$$

where δ_{ij} is the Kronecker delta. Therefore,

$$\left\langle u_i v_i^\top, u_j v_j^\top \right\rangle_F = \delta_{ij}. \tag{23}$$

Thus, the rank-one SVD components are orthonormal under the Frobenius inner product. Since

$$A - A_r = \sum_{i=r+1}^{p} \sigma_i u_i v_i^\top,$$

the Pythagorean theorem for the Frobenius norm gives

$$\|A - A_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2. \tag{24}$$

This proves that A_r achieves the error claimed in the theorem. It remains to show that no other matrix of rank at most r can do better.

Step 2: Consider an arbitrary rank-r approximation. Let

$$B \in \mathbb{R}^{m \times n}$$

be any matrix satisfying

$$\operatorname{rank}(B) \le r.$$

Let $S \subseteq \mathbb{R}^n$ be the row space of B, and let

$$q = \dim(S) = \operatorname{rank}(B) \le r.$$

Let $P \in \mathbb{R}^{n \times n}$ be the orthogonal projection onto S. Thus,

$$P^\top = P, \qquad P^2 = P.$$

Every row of B belongs to S, so projecting the rows of B onto S leaves them unchanged. Therefore,

$$B = BP. \tag{25}$$

We can now decompose the approximation error as

$$A - B = A(I - P) + AP - B. \tag{26}$$

Every row of $A(I - P)$ lies in $S^{\perp}$, while every row of $AP - B$ lies in S. Indeed,

$$AP - B = AP - BP = (A - B)P,$$

so its rows belong to the image of P, which is S. Therefore, the two terms in Equation (26) are orthogonal under the Frobenius inner product:

$$\langle A(I - P), AP - B \rangle_F = 0.$$

Applying the Pythagorean theorem gives

$$\|A - B\|_F^2 = \|A(I - P)\|_F^2 + \|AP - B\|_F^2. \tag{27}$$

Since the second term is nonnegative,

$$\|A - B\|_F^2 \geq \|A(I - P)\|_F^2. \tag{28}$$

Consequently, it is enough to determine the smallest amount of Frobenius energy that must remain outside any subspace of dimension at most r.

Step 3: Bound the energy captured by an r-dimensional subspace. The decomposition

$$A = AP + A(I - P)$$

is also an orthogonal decomposition under the Frobenius inner product. Therefore,

$$\|A(I - P)\|_F^2 = \|A\|_F^2 - \|AP\|_F^2. \tag{29}$$

We now bound the captured energy $\|AP\|_F^2$. Using $P^{\top} = P$, $P^2 = P$, and the cyclic property of the trace,

$$\begin{aligned} \|AP\|_F^2 &= \operatorname{tr}\left((AP)^{\top}(AP)\right) && (30)\\ &= \operatorname{tr}\left(PA^{\top}AP\right) && (31)\\ &= \operatorname{tr}\left(A^{\top}AP\right). && (32) \end{aligned}$$

From the singular value decomposition,

$$A^{\top}A = \sum_{i=1}^{p} \sigma_i^2 v_i v_i^{\top}. \tag{33}$$

Substituting this into Equation (32) gives

$$\begin{aligned} \|AP\|_F^2 &= \sum_{i=1}^{p} \sigma_i^2 \operatorname{tr}\left(v_i v_i^{\top} P\right) && (34)\\ &= \sum_{i=1}^{p} \sigma_i^2 v_i^{\top} P v_i. && (35) \end{aligned}$$

Because P is an orthogonal projection,

$$v_i^\top P v_i = \|P v_i\|_2^2.$$

Define

$$\alpha_i = \|P v_i\|_2^2.$$

Then

$$\|AP\|_F^2 = \sum_{i=1}^{p} \sigma_i^2 \alpha_i. \tag{36}$$

Since an orthogonal projection cannot increase Euclidean length,

$$0 \le \alpha_i \le 1.$$

Furthermore, since P has rank q,

$$\sum_{i=1}^{p} \alpha_i \le q.$$

To see this, extend $v_1, \dots, v_p$ to an orthonormal basis of $\mathbb{R}^n$. The sum of $v^\top P v$ over the entire basis equals $\operatorname{tr}(P) = q$, and every term is nonnegative.

Because

$$\sigma_1^2 \ge \sigma_2^2 \ge \cdots \ge \sigma_p^2,$$

the sum in Equation (36) is maximized by placing as much of the available weight as possible on the largest singular values.

For completeness, suppose $q \ge 1$. Then

$$\begin{aligned}
\sum_{i=1}^{p} \sigma_i^2 \alpha_i &= \sum_{i=1}^{q} \sigma_i^2 \alpha_i + \sum_{i=q+1}^{p} \sigma_i^2 \alpha_i && (37)\\
&\le \sum_{i=1}^{q} \sigma_i^2 \alpha_i + \sigma_q^2 \sum_{i=q+1}^{p} \alpha_i && (38)\\
&\le \sum_{i=1}^{q} \sigma_i^2 \alpha_i + \sigma_q^2 \left(q - \sum_{i=1}^{q} \alpha_i \right) && (39)\\
&= q\sigma_q^2 + \sum_{i=1}^{q} (\sigma_i^2 - \sigma_q^2)\alpha_i && (40)\\
&\le q\sigma_q^2 + \sum_{i=1}^{q} (\sigma_i^2 - \sigma_q^2) && (41)\\
&= \sum_{i=1}^{q} \sigma_i^2. && (42)
\end{aligned}$$

When $q = 0$, $P = 0$, so the same conclusion is immediate. Since $q \leq r$, it follows that

$$\|AP\|_F^2 \leq \sum_{i=1}^{q} \sigma_i^2 \leq \sum_{i=1}^{r} \sigma_i^2. \tag{43}$$

Thus, no subspace of dimension at most r can capture more than

$$\sum_{i=1}^{r} \sigma_i^2$$

of the Frobenius energy of A.

Step 4: Obtain the lower bound for every rank-r matrix. Since the squared Frobenius norm of A is

$$\|A\|_F^2 = \sum_{i=1}^{p} \sigma_i^2, \tag{44}$$

Equations (29) and (43) imply

$$\|A(I - P)\|_F^2 = \|A\|_F^2 - \|AP\|_F^2 \tag{45}$$

$$\geq \sum_{i=1}^{p} \sigma_i^2 - \sum_{i=1}^{r} \sigma_i^2 \tag{46}$$

$$= \sum_{i=r+1}^{p} \sigma_i^2. \tag{47}$$

Combining this with Equation (28) gives

$$\|A - B\|_F^2 \geq \sum_{i=r+1}^{p} \sigma_i^2 \tag{48}$$

for every matrix B satisfying $\operatorname{rank}(B) \leq r$.

On the other hand, Equation (24) shows that A_r satisfies

$$\|A - A_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2.$$

Therefore, A_r attains the lower bound and is an optimal rank-r approximation of A. Taking square roots proves Equation (18). □

3.1 Geometric Interpretation

The right singular vectors

$$v_1, \ldots, v_p$$

represent mutually orthogonal directions in the input space $\mathbb{R}^n$. The action of A on each of these directions is

$$Av_i = \sigma_i u_i. \tag{49}$$

Thus, A maps the input direction v_i to the output direction u_i and scales its length by σ_i.

For an arbitrary input $x \in \mathbb{R}^n$,

$$Ax = \sum_{i=1}^{p} \sigma_i u_i (v_i^\top x). \tag{50}$$

The scalar $v_i^\top x$ measures the component of x in the direction v_i. The truncated matrix acts as

$$A_r x = \sum_{i=1}^{r} \sigma_i u_i (v_i^\top x). \tag{51}$$

Therefore, A_r preserves the action of A along the first r right singular directions and discards its action along the remaining directions.

Let

$$P_r = V_r V_r^\top$$

be the orthogonal projection onto

$$\mathcal{V}_r = \operatorname{span}\{v_1, \ldots, v_r\}.$$

Then

$$A_r = AP_r. \tag{52}$$

Thus, geometrically, the truncated SVD first projects the input onto the subspace spanned by the r most important right singular vectors and then applies A to that projected input.

The squared Frobenius norm can also be written as

$$\|A - B\|_F^2 = \sum_{j=1}^{n} \|(A - B)e_j\|_2^2, \tag{53}$$

where $e_1, \ldots, e_n$ are the standard basis vectors of $\mathbb{R}^n$. Therefore, the Frobenius norm measures the total squared discrepancy between the actions of A and B across an orthonormal basis of input directions.

A rank-r matrix can use at most r independent input directions through its row space. The Eckart–Young theorem states that the optimal choice is the subspace

$$\operatorname{span}\{v_1, \ldots, v_r\}.$$

This subspace captures the largest possible amount of the total action, or energy, of A. Any other r-dimensional subspace leaves at least as much energy unexplained.

3.2 Tail-Singular-Value Error Formula

Define the optimal rank-r approximation error by

$$E_r = \min_{\operatorname{rank}(B)\le r} \|A - B\|_F.$$

The Eckart–Young theorem gives the exact formula

$$E_r = \left(\sum_{i=r+1}^{p} \sigma_i^2 \right)^{1/2}. \tag{54}$$

Equivalently,

$$E_r^2 = \sum_{i=r+1}^{p} \sigma_i^2. \tag{55}$$

This is called the tail-singular-value formula because the approximation error is determined exactly by the singular values that are omitted from the truncated SVD. Each discarded singular component contributes σ_i^2 independently to the squared error.

In particular,

$$E_{r-1}^2 - E_r^2 = \sigma_r^2. \tag{56}$$

Thus, adding the r-th singular component reduces the squared approximation error by exactly σ_r^2.

Since

$$\|A\|_F^2 = \sum_{i=1}^{p} \sigma_i^2,$$

the relative squared approximation error is

$$\frac{\|A - A_r\|_F^2}{\|A\|_F^2} = \frac{\sum_{i=r+1}^{p} \sigma_i^2}{\sum_{i=1}^{p} \sigma_i^2}. \tag{57}$$

Correspondingly, the fraction of Frobenius energy retained by the rank-r approximation is

$$\frac{\|A_r\|_F^2}{\|A\|_F^2} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{p} \sigma_i^2}. \tag{58}$$

These formulas explain why the decay of the singular values is central to low-rank approximation. If the singular values decrease rapidly, then the tail sum becomes small for a relatively small value of r, and A can be approximated accurately by a low-rank matrix. If the singular values decrease slowly, then a larger rank is required to obtain the same approximation accuracy.

For a neural-network weight matrix, the Eckart–Young theorem implies that replacing A by A_r produces the smallest possible Frobenius-norm perturbation among all matrices of rank at most r. Furthermore, the factorization

$$A_r = (U_r \Sigma_r)\, V_r^\top$$

can be implemented as two consecutive linear transformations,

$$\mathbb{R}^n \longrightarrow \mathbb{R}^r \longrightarrow \mathbb{R}^m.$$

When r is small, these two factors require

$$nr + mr = r(m + n)$$

stored parameters rather than the mn parameters required by the original dense matrix. This is the mathematical basis for using truncated SVD as a method for compressing linear layers in neural networks.

4 Low-Rank Neural-Network Matrices

Large neural networks often contain weight matrices with millions of parameters. In particular, fully connected layers can account for a substantial portion of a model's storage and computational cost. Low-rank factorization provides a mathematically principled way to replace one large weight matrix with two smaller matrices, thereby reducing the number of parameters while approximately preserving the action of the original layer [1, 6, 8].

4.1 Dense Layers

Let

$$x \in \mathbb{R}^{d_{\text{in}}}$$

be the input to a fully connected, or dense, neural-network layer. Before applying a nonlinear activation function, the layer computes

$$z = Wx + b,$$

where

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}} \qquad \text{and} \qquad b \in \mathbb{R}^{d_{\text{out}}}.$$

If the layer includes an activation function ϕ, its final output is

$$y = \phi(z) = \phi(Wx + b).$$

If no activation function is present, then ϕ is the identity function and $y = Wx + b$. Such affine transformations are basic building blocks of feedforward neural networks [4].

The matrix W contains one weight for every input–output connection. Consequently, the number of weight parameters in the dense layer is

$$P_{\text{dense}}^{(W)} = d_{\text{out}} d_{\text{in}}.$$

The bias vector contributes an additional d_{out} parameters, so the total number of trainable parameters is

$$P_{\text{dense}} = d_{\text{out}} d_{\text{in}} + d_{\text{out}}.$$

When both d_{in} and d_{out} are large, the product $d_{\text{out}} d_{\text{in}}$ can dominate the parameter count of the network. This motivates replacing W with a more compact representation.

4.2 Two-Factor Low-Rank Representation

Suppose that the weight matrix W is exactly or approximately low rank. Instead of storing W directly, we approximate it by a product

$$W \approx BA,$$

where

$$A \in \mathbb{R}^{r \times d_{\text{in}}} \qquad \text{and} \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

Here,

$$1 \leq r \leq \min(d_{\text{in}}, d_{\text{out}})$$

is the chosen approximation rank.

The original dense transformation

$$x \longmapsto Wx + b$$

is then replaced by the two-stage transformation

$$h = Ax, \qquad z = Bh + b.$$

Equivalently,

$$z = B(Ax) + b.$$

The intermediate vector

$$h \in \mathbb{R}^r$$

can be interpreted as an r-dimensional bottleneck. The first matrix A maps the input into this lower-dimensional space, while the second matrix B maps the compressed representation into the output space.

Because

$$\operatorname{rank}(BA) \leq \min\big(\operatorname{rank}(B), \operatorname{rank}(A)\big) \leq r,$$

the resulting weight matrix has rank at most r. If $\operatorname{rank}(W) \leq r$, then an exact factorization $W = BA$ is possible. Otherwise, BA is an approximation of W.

No nonlinear activation function should be inserted between A and B if the goal is to implement the same low-rank linear transformation. In other words, the correct computation is

$$y = \phi\big(B(Ax) + b\big),$$

not

$$y = \phi\big(B\phi(Ax) + b\big).$$

Adding a nonlinear activation between the two factors would produce a different neural-network architecture rather than a factorization of the original dense layer.

4.3 Parameter Count of the Factorized Layer

The first factor

$$A \in \mathbb{R}^{r \times d_{\text{in}}}$$

contains

$$r d_{\text{in}}$$

weight parameters. The second factor

$$B \in \mathbb{R}^{d_{\text{out}} \times r}$$

contains

$$d_{\text{out}} r$$

weight parameters. Therefore, the factorized representation contains

$$P^{(W)}_{\text{factored}} = r d_{\text{in}} + d_{\text{out}} r = r(d_{\text{in}} + d_{\text{out}})$$

weight parameters.

The original bias vector b is retained after the second transformation. Thus, including the bias, the total parameter count becomes

$$P_{\text{factored}} = r(d_{\text{in}} + d_{\text{out}}) + d_{\text{out}}.$$

The bias should not be added after the first transformation. In a neural network implementation, the factorization should therefore consist of a bias-free layer from d_{in} dimensions to r dimensions, followed by a layer from r dimensions to d_{out} dimensions that contains the original bias b.

4.4 Condition for Parameter Compression

A low-rank factorization reduces the number of weight parameters only when

$$P^{(W)}_{\text{factored}} < P^{(W)}_{\text{dense}}.$$

Substituting the parameter-count formulas gives

$$r(d_{\text{in}} + d_{\text{out}}) < d_{\text{in}} d_{\text{out}}.$$

Solving for r, compression occurs precisely when

$$\boxed{r < \frac{d_{\text{in}} d_{\text{out}}}{d_{\text{in}} + d_{\text{out}}}}.$$

The bias vector appears in both representations and therefore cancels when the two total parameter counts are compared:

$$\begin{aligned} P_{\text{dense}} - P_{\text{factored}} &= (d_{\text{in}} d_{\text{out}} + d_{\text{out}}) - (r(d_{\text{in}} + d_{\text{out}}) + d_{\text{out}}) \\ &= d_{\text{in}} d_{\text{out}} - r(d_{\text{in}} + d_{\text{out}}). \end{aligned}$$

Thus, the same compression condition is obtained whether or not the unchanged bias is included in the parameter count.

At equality,

$$r = \frac{d_{\text{in}} d_{\text{out}}}{d_{\text{in}} + d_{\text{out}}},$$

the two representations contain the same number of weight parameters. If r is above this threshold, the factorized representation is actually larger than the original dense matrix, even though its product has rank at most r.

For a square weight matrix with

$$d_{\text{in}} = d_{\text{out}} = d,$$

the compression condition simplifies to

$$r < \frac{d^2}{2d} = \frac{d}{2}.$$

Therefore, a factorization of a $d \times d$ weight matrix reduces the parameter count only when its rank is strictly less than one half of the matrix dimension.

The fraction of weight parameters retained after factorization is

$$\frac{P^{(W)}_{\text{factored}}}{P^{(W)}_{\text{dense}}} = \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}.$$

Consequently, the fraction of weight parameters removed is

$$1 - \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}.$$

Expressed as a percentage, the weight-parameter reduction is

$$100 \left(1 - \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}\right) \%.$$

Including the unchanged bias, the exact fraction of total layer parameters retained is

$$\frac{r(d_{\text{in}} + d_{\text{out}}) + d_{\text{out}}}{d_{\text{in}} d_{\text{out}} + d_{\text{out}}}.$$

4.5 Constructing the Factors Using Truncated SVD

Suppose that the singular value decomposition of the trained weight matrix is

$$W = U \Sigma V^\top.$$

For a chosen rank r, the truncated SVD approximation is

$$W_r = U_r \Sigma_r V_r^\top,$$

where U_r and V_r contain the first r left and right singular vectors, respectively, and Σ_r contains the first r singular values.

A valid two-factor representation of W_r is obtained by defining

$$A = V_r^\top \qquad \text{and} \qquad B = U_r \Sigma_r.$$

The dimensions of these factors are

$$A \in \mathbb{R}^{r \times d_{\text{in}}} \qquad \text{and} \qquad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

and their product is

$$BA = (U_r \Sigma_r) V_r^\top = U_r \Sigma_r V_r^\top = W_r.$$

The factorized forward pass is therefore

$$x \longmapsto V_r^\top x \longmapsto U_r \Sigma_r (V_r^\top x) + b.$$

In terms of the two intermediate computations,

$$h = V_r^\top x$$

and

$$z = U_r \Sigma_r h + b.$$

The original bias b is included only after the second transformation.

By the Eckart–Young theorem, W_r is the best rank-r approximation of W in the Frobenius norm. Its approximation error is

$$\|W - W_r\|_F = \left(\sum_{i=r+1}^{p} \sigma_i^2 \right)^{1/2}, \qquad p = \min(d_{\text{in}}, d_{\text{out}}).$$

Thus, the compression threshold and the approximation error answer two different questions. The inequality

$$r < \frac{d_{\text{in}} d_{\text{out}}}{d_{\text{in}} + d_{\text{out}}}$$

determines whether the factorized representation contains fewer parameters, whereas the tail singular values determine how accurately that representation approximates the original weight matrix.

4.6 Important Implementation Distinction

It is not sufficient to compute the truncated matrix W_r and then store W_r as another ordinary dense matrix. Although W_r has rank at most r, it still has dimensions

$$d_{\text{out}} \times d_{\text{in}},$$

so a conventional dense representation of W_r still stores

$$d_{\text{out}} d_{\text{in}}$$

scalar entries. In that case, no parameter reduction has been realized.

To obtain actual compression, the original dense layer must be replaced by the two smaller transformations

$$x \mapsto Ax \mapsto B(Ax) + b,$$

and the original matrix W must no longer be stored as part of the model. The first transformation should have no bias, and the second transformation should retain the original bias.

For a single input vector, the dense matrix multiplication requires on the order of

$$d_{\text{in}} d_{\text{out}}$$

scalar multiplications. The factorized computation requires on the order of

$$r d_{\text{in}} + d_{\text{out}} r = r(d_{\text{in}} + d_{\text{out}})$$

multiplications. Therefore, the same inequality that gives parameter compression also gives a reduction in the theoretical arithmetic cost. Actual wall-clock speedups, however, can depend on hardware, batch size, memory access, and the efficiency of the matrix-multiplication implementation.

Consequently, a matrix should not be described as "compressed" merely because a rank-r approximation has been computed. Parameter compression is achieved only when the rank satisfies the compression threshold and the model stores and evaluates the two smaller factors rather than the full $d_{\text{out}} \times d_{\text{in}}$ matrix.

5 Truncated SVD versus LoRA

Truncated singular value decomposition and Low-Rank Adaptation (LoRA) both use low-rank matrix factorizations, but they solve fundamentally different problems. Truncated SVD is a *post hoc matrix-approximation method*: it starts with an already known matrix and constructs the best rank-r approximation to that matrix under a specified matrix norm. LoRA, in contrast, is a *parameter-efficient adaptation method*: it keeps a pretrained matrix fixed and learns a low-rank update by minimizing a downstream task loss. Although both methods may represent a matrix using two smaller factors, their starting points, optimization objectives, and interpretations are different.

5.1 Truncated SVD as Post Hoc Approximation

Suppose that a neural-network layer has already been trained and has weight matrix

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}.$$

Let the singular value decomposition of W be

$$W = U\Sigma V^\top = \sum_{i=1}^{p} \sigma_i u_i v_i^\top, \qquad p = \min(d_{\text{out}}, d_{\text{in}}),$$

where

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_p \geq 0.$$

The rank-r truncated SVD is

$$W_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top = U_r \Sigma_r V_r^\top.$$

By the Eckart–Young theorem, W_r solves the rank-constrained matrix-approximation problem

$$W_r \in \underset{\text{rank}(\widetilde{W}) \leq r}{\text{argmin}} \left\| W - \widetilde{W} \right\|_F.$$

The corresponding approximation error is

$$\|W - W_r\|_F^2 = \sum_{i=r+1}^{p} \sigma_i^2.$$

Thus, truncated SVD is optimal for reconstructing the entries of W under the Frobenius norm [2, 3].

The approximation can be implemented as a two-layer factorization. For example, define

$$A_{\text{SVD}} = V_r^\top \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B_{\text{SVD}} = U_r \Sigma_r \in \mathbb{R}^{d_{\text{out}} \times r}.$$

Then

$$W_r = B_{\text{SVD}} A_{\text{SVD}},$$

and the original dense layer

$$y = Wx + b$$

is replaced by

$$y \approx W_r x + b = B_{\text{SVD}} \left(A_{\text{SVD}} x \right) + b.$$

This procedure is described as *post hoc* because it is applied after the full matrix W has already been obtained through training. No task data are required to compute W_r once W is known, and the solution is available directly from the SVD rather than through another neural-network training procedure.

However, the Eckart–Young guarantee concerns matrix reconstruction, not predictive performance. In particular, minimizing

$$\left\| W - \widetilde{W} \right\|_F$$

does not necessarily minimize the change in classification accuracy, cross-entropy loss, or another downstream performance metric. A singular direction associated with a relatively small singular value contributes little to the Frobenius norm but may still be important for particular inputs or classes. Therefore, a small matrix-approximation error does not automatically imply a small task-performance error.

5.2 LoRA as a Learned Low-Rank Update

LoRA begins from a different setting. Let

$$W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$$

be a pretrained weight matrix. Instead of replacing W_0 by a low-rank approximation, LoRA freezes W_0 and learns a low-rank change to it. The update is parameterized as

$$\Delta W = BA,$$

where

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r}.$$

Because

$$\operatorname{rank}(BA) \leq r,$$

the learned update is constrained to have rank at most r.

The effective weight matrix is commonly written as

$$W_{\text{effective}} = W_0 + \frac{\alpha}{r} BA,$$

where α is a scaling hyperparameter. The corresponding forward pass is

$$y = W_0 x + \frac{\alpha}{r} BAx + b.$$

During training, W_0 remains fixed, while the entries of A and B are learned [5].

For a downstream dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N},$$

LoRA approximately solves an optimization problem of the form

$$\underset{A,B}{\text{minimize}} \quad \mathcal{L}_{\text{task}} \left(W_0 + \frac{\alpha}{r} BA; \mathcal{D} \right).$$

For example, for supervised learning this objective may be written as

$$\underset{A,B}{\text{minimize}} \quad \frac{1}{N} \sum_{i=1}^{N} \ell \left(f \left(x_i; W_0 + \frac{\alpha}{r} BA \right), y_i \right),$$

where f denotes the neural network and ℓ is a task-dependent loss function such as cross-entropy. In a complete neural network, LoRA factors may be inserted into several weight matrices simultaneously, and all of these factors are trained jointly by gradient-based optimization.

The number of trainable weight parameters associated with this update is

$$r \left(d_{\text{in}} + d_{\text{out}} \right),$$

rather than

$$d_{\text{in}} d_{\text{out}}$$

parameters for unrestricted fine-tuning of the entire matrix. The pretrained matrix W_0, however, is still retained. Consequently, standard LoRA primarily reduces the number of *trainable* and *task-specific* parameters; it does not, by itself, replace the base matrix with a smaller compressed matrix.

This distinction can also be expressed in terms of rank. LoRA guarantees only that

$$\operatorname{rank}(\Delta W) = \operatorname{rank}(BA) \leq r.$$

It does not guarantee that the effective matrix has rank at most r. Indeed,

$$\operatorname{rank}\left(W_0 + \frac{\alpha}{r} BA\right) \leq \operatorname{rank}(W_0) + r,$$

and if W_0 is full rank, the effective matrix will generally remain full rank. The low-rank restriction is placed on the *change to the model*, not on the entire effective weight matrix.

After training, the LoRA update can be merged into the base matrix by defining

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} BA.$$

This produces an ordinary dense matrix for inference. Alternatively, the factors A and B can be stored separately, allowing the same pretrained model to be shared across multiple tasks while storing only a small pair of task-specific matrices for each task.

5.3 Matrix Objective versus Task Objective

The central mathematical distinction is visible by comparing the two optimization problems. Truncated SVD solves

$$\underset{\operatorname{rank}(\widetilde{W}) \leq r}{\text{minimize}} \quad \left\| W - \widetilde{W} \right\|_F^2.$$

Here, the target is the known matrix W, and the objective measures distance in matrix space. The resulting approximation is determined entirely by the singular values and singular vectors of W.

LoRA instead solves an objective of the form

$$\underset{A,B}{\text{minimize}} \quad \mathcal{L}_{\text{task}}\left(W_0 + \frac{\alpha}{r} BA; \mathcal{D}\right).$$

Here, there is generally no known target matrix that BA is attempting to reconstruct. The factors are selected according to how they affect the outputs of the complete neural network on the downstream dataset.

Therefore, the two procedures define different notions of optimality:

- Truncated SVD is optimal with respect to reconstruction of a known matrix under the Frobenius norm.

- LoRA is optimized with respect to a task-dependent loss evaluated through the complete neural network.

These objectives can produce different low-rank directions. Truncated SVD preserves the directions that account for the greatest squared magnitude in the original matrix. LoRA instead learns the directions that most effectively improve the downstream task, even if those directions do not correspond to the largest singular directions of W_0.

5.4 Why LoRA Is Not Simply the Truncated SVD of an Update

One possible alternative adaptation procedure would be to first perform unrestricted fine-tuning, producing

$$W_{\text{full}} = W_0 + \Delta W_{\text{full}},$$

and then approximate the full update by solving

$$\underset{\text{rank}(\widetilde{\Delta W})\leq r}{\text{minimize}} \quad \left\| \Delta W_{\text{full}} - \widetilde{\Delta W} \right\|_F .$$

The solution would be the truncated SVD of ΔW_{full}. This would be a post hoc compression of a previously computed fine-tuning update.

Standard LoRA does not follow this procedure. It does not first compute an unrestricted update and then approximate it. Instead, the low-rank parameterization is imposed from the beginning, and the factors A and B are learned directly through the downstream task loss. Therefore, a LoRA update is not generally guaranteed to equal the truncated SVD of an unrestricted fine-tuning update. Even when both methods use the same rank r, one minimizes matrix reconstruction error while the other minimizes task loss.

5.5 Summary of the Distinction

Table 1 summarizes the principal differences between the two methods.

The essential distinction can therefore be summarized as follows:

> **Truncated SVD finds the best low-rank representation of a known matrix under a matrix norm, whereas LoRA learns a low-rank update directly to minimize a task-dependent objective.**

This distinction is important when interpreting experimental results. A truncated-SVD experiment measures how well a trained weight matrix can be approximated after training and how that approximation affects model performance. A LoRA experiment instead measures whether a model can adapt to a task while restricting the trainable update to a low-dimensional family. Thus, truncated SVD evaluates post hoc compressibility, whereas LoRA evaluates parameter-efficient task adaptation.

Property	**Truncated SVD**	**LoRA**
Starting point	An already trained, known matrix W	A frozen pretrained matrix W_0 and downstream training data
Operation	Replaces W by a low-rank approximation W_r	Adds a learned low-rank update $\frac{\alpha}{r}BA$ to W_0
Low-rank object	The approximating matrix W_r	The update $\Delta W = BA$
Objective	Minimize $\|W - \widetilde{W}\|_F$	Minimize a downstream task loss
Optimization method	Closed-form construction from the SVD	Gradient-based learning of A and B
Guarantee	Best rank-r matrix reconstruction under the Frobenius norm	No general closed-form matrix-approximation guarantee
Effect on base matrix	The original matrix is replaced or approximated	The pretrained matrix remains frozen
Parameter benefit	Can reduce the stored and executed parameters of the layer when factors are retained	Reduces trainable and task-specific parameters while retaining the base model
Task awareness	Does not directly use task labels or task loss	Learns directly from the downstream task objective

Table 1: Comparison of truncated-SVD compression and Low-Rank Adaptation.

6 Experimental Methodology

The computational component was designed to connect the Eckart–Young theorem to the behavior of a trained neural network. The experiment first trained a dense image classifier on Fashion–MNIST, then examined the singular values of two hidden-layer weight matrices, replaced those matrices by truncated-SVD factorizations of different ranks, and measured both matrix-reconstruction quality and classification performance. An adaptation experiment subsequently compared post hoc SVD compression of a full fine-tuning update with a directly learned LoRA update.

6.1 Dataset and Preprocessing

The experiments used the Fashion–MNIST dataset, which consists of 28×28 grayscale images from ten clothing categories. The official training set contains 60,000 images, while the official test set contains 10,000 images. A deterministic random split divided the original training set into 54,000 training examples and 6,000 validation examples. The official 10,000-image test set remained completely separate from training, checkpoint selection, and learning-rate selection.

For the source classification task, each image was converted to a tensor and normalized channel-wise according to

$$x_{\text{normalized}} = \frac{x - 0.5}{0.5}. \tag{59}$$

Because the original pixel values lie in $[0, 1]$, this transformation maps them approximately to $[-1, 1]$.

The adaptation experiment used a deterministic transformed version of Fashion–MNIST. Each image was rotated by $+20^\circ$ using bilinear interpolation, without expanding the image canvas, and empty pixels were filled with zero. The rotated images were then normalized using the same mean and standard deviation as the source images. The class labels and the train–validation indices were unchanged, so the source and target datasets differed only through the controlled image transformation.

6.2 Model Architecture and Layer Selection

The source classifier was a fully connected neural network with architecture

$$784 \longrightarrow 256 \longrightarrow 128 \longrightarrow 10. \tag{60}$$

A 28×28 input image was flattened to a vector in $\mathbb{R}^{784}$. The first two affine transformations were followed by ReLU activations, while the final layer produced ten unnormalized class logits:

$$h_1 = \mathrm{ReLU}(W_1 x + b_1), \tag{61}$$

$$h_2 = \mathrm{ReLU}(W_2 h_1 + b_2), \tag{62}$$

$$z = W_3 h_2 + b_3. \tag{63}$$

The corresponding weight matrices had dimensions

$$W_1 \in \mathbb{R}^{256\times 784}, \qquad W_2 \in \mathbb{R}^{128\times 256}, \qquad W_3 \in \mathbb{R}^{10\times 128}. \tag{64}$$

The complete dense model contained 235,146 parameters. Of these, W_1 contained 200,704 weight parameters, W_2 contained 32,768, and the two selected matrices together contained 233,472 weight parameters. The remaining 1,674 parameters consisted of the three bias vectors and the final-layer weight matrix.

The compression experiments focused on W_1 and W_2. The final matrix W_3 was excluded because its maximum possible rank is only 10, which makes it less informative for a broad rank sweep.

6.3 Baseline Training Procedure

The model was trained using cross-entropy loss and the Adam optimizer. The source-training configuration was

$$\text{batch size} = 128, \qquad \text{learning rate} = 10^{-3}, \qquad \text{weight decay} = 0, \tag{65}$$

with a maximum training budget of 20 epochs. Training computations used 32-bit floating-point arithmetic.

A validation evaluation was performed after every epoch. Training continued for the full epoch budget, but the checkpoint with the lowest validation cross-entropy loss was saved as the final source model. Thus, checkpoint selection depended only on validation loss and did not use the official test set. The selected source checkpoint occurred at epoch 9.

For the target-domain adaptation experiments, both unrestricted fine-tuning and selected-layer full-rank adaptation were trained for 15 epochs using Adam with learning rate 10^{-3}, batch size 128, and zero weight decay. Once again, the checkpoint with the smallest target-validation loss was retained.

6.4 Singular-Value Analysis

After training, the selected source weight matrices were copied from the saved checkpoint and converted to 64-bit floating-point arithmetic for singular-value and reconstruction calculations. For each matrix W, the reduced singular value decomposition

$$W = U\Sigma V^\top \tag{66}$$

was computed. All singular values were saved, along with their normalized values

$$\frac{\sigma_i}{\sigma_1} \tag{67}$$

and cumulative retained energy

$$E_r = \frac{\sum_{i=1}^{r}\sigma_i^2}{\sum_{i=1}^{p}\sigma_i^2}, \qquad p = \min(d_{\text{in}}, d_{\text{out}}). \tag{68}$$

The smallest ranks retaining 90%, 95%, and 99% of the Frobenius energy were then determined separately for W_1 and W_2.

6.5 Truncated-SVD Compression Procedure

For each selected rank r, the truncated-SVD approximation was

$$W_r = U_r \Sigma_r V_r^\top. \tag{69}$$

The approximation was implemented as two consecutive linear transformations. The factors were constructed symmetrically as

$$A_r = \Sigma_r^{1/2} V_r^\top, \qquad B_r = U_r \Sigma_r^{1/2}, \tag{70}$$

so that

$$B_r A_r = U_r \Sigma_r V_r^\top = W_r. \tag{71}$$

The first factor had no bias, while the second factor received the original dense-layer bias. No activation function was inserted between the two factors. Therefore, the factorized layer represented exactly the same linear transformation as the reconstructed matrix W_r.

Every compressed model was created from a fresh copy of the canonical baseline checkpoint. All factorized parameters were frozen, and no retraining or post-compression fine-tuning was performed. This design isolated the immediate effect of replacing the dense matrix by its best rank-r Frobenius approximation.

Three principal compression configurations were evaluated:

1. W_1-only compression, with W_2 and W_3 unchanged;
2. W_2-only compression, with W_1 and W_3 unchanged;
3. joint compression of W_1 and W_2.

For the joint experiment, most configurations used the same rank for both layers. Additional configurations assigned different ranks to the layers so that each retained approximately 90%, 95%, or 99% of its own singular-value energy.

6.6 Rank Values

The source-weight rank grids combined fixed ranks with the ranks identified by the energy analysis. For W_1-only compression, the evaluated ranks were

$$\begin{aligned} r \in \{&1, 2, 4, 8, 16, 32, 48, 64, 80, 82, 96, 112, 128, 132, \\ &160, 192, 212, 224, 256\}. \end{aligned} \tag{72}$$

For W_2-only compression, the evaluated ranks were

$$r \in \{1, 2, 4, 8, 16, 32, 48, 61, 64, 80, 81, 85, 96, 110, 112, 128\}. \tag{73}$$

For joint same-rank compression, the evaluated ranks were

$$r \in \{1, 2, 4, 8, 16, 32, 48, 61, 64, 80, 81, 82, 85, 96, 110, 112, 128\}. \tag{74}$$

The layer-specific energy-matched pairs were

$$(r_1, r_2) = (82, 61), \qquad (132, 81), \qquad (212, 110), \tag{75}$$

corresponding approximately to 90%, 95%, and 99% retained energy. The exact-rank pair $(256, 128)$ was also evaluated as a numerical control.

6.7 Evaluation Metrics

For each layer and rank, the absolute reconstruction error was

$$\epsilon_{\text{abs}}(r) = \|W - W_r\|_F, \tag{76}$$

and the relative reconstruction error was

$$\epsilon_{\text{rel}}(r) = \frac{\|W - W_r\|_F}{\|W\|_F}. \tag{77}$$

The theoretical Eckart–Young error was calculated directly from the discarded singular values:

$$\epsilon_{\text{EY}}(r) = \sqrt{\sum_{i=r+1}^{p} \sigma_i^2}. \tag{78}$$

The directly computed Frobenius error and the theoretical tail-singular-value error were compared numerically. For an exact truncated SVD, retained energy and relative error satisfy

$$E_r = 1 - \epsilon_{\text{rel}}(r)^2. \tag{79}$$

For joint compression, a combined relative reconstruction error was defined as

$$\epsilon_{\text{joint}} = \frac{\sqrt{\|W_1 - \widehat{W}_1\|_F^2 + \|W_2 - \widehat{W}_2\|_F^2}}{\sqrt{\|W_1\|_F^2 + \|W_2\|_F^2}}. \tag{80}$$

Each model was evaluated using validation and test cross-entropy loss and classification accuracy. The test-accuracy drop was reported in percentage points:

$$\Delta_{\text{acc}} = 100\,(a_{\text{baseline}} - a_{\text{compressed}})\,. \tag{81}$$

A negative value means that the compressed model was slightly above the baseline in this single deterministic run.

For parameter accounting, a rank-r_1 factorization of W_1 contains

$$r_1(784 + 256) = 1040r_1 \tag{82}$$

parameters, while a rank-r_2 factorization of W_2 contains

$$r_2(256 + 128) = 384r_2 \tag{83}$$

parameters. When both layers are factorized, the whole-model parameter count is therefore

$$N_{\text{joint}}(r_1, r_2) = 1040r_1 + 384r_2 + 1674, \tag{84}$$

where 1,674 accounts for the unchanged biases and output layer. For equal ranks,

$$N_{\text{joint}}(r) = 1424r + 1674. \tag{85}$$

To avoid ambiguity, two separate compression quantities were reported. The parameter fraction was

$$q = \frac{N_{\text{compressed}}}{N_{\text{dense}}}, \tag{86}$$

while the compression factor was

$$c = \frac{N_{\text{dense}}}{N_{\text{compressed}}} = \frac{1}{q}. \tag{87}$$

A smaller parameter fraction and a larger compression factor indicate stronger compression.

The factorization is smaller than the original W_1 only for $r_1 \leq 192$, and smaller than the original W_2 only for $r_2 \leq 85$. Joint equal-rank factorization reduces the whole-model parameter count only for $r \leq 163$. Thus, high-rank factorizations were retained as numerical controls but were not described as compressed models.

6.8 LoRA Adaptation Experiment

6.8.1 Target Task and Full-Rank References

The LoRA experiment treated the source checkpoint as the pretrained model W_0 and the $+20^\circ$ rotated dataset as the target task. Three reference evaluations were produced:

1. zero-shot evaluation of W_0 on the rotated data;
2. unrestricted fine-tuning of all 235,146 model parameters;
3. layer-matched full-rank adaptation in which only W_1 and W_2 were trainable.

The layer-matched model was the principal reference for the SVD-versus-LoRA comparison. Its trainable set contained exactly 233,472 parameters. All biases and the output-layer matrix W_3 were frozen, matching the locations at which the low-rank adapters were applied. Unrestricted fine-tuning was included as a contextual full-model reference rather than as the rank-matched comparator.

For each selected layer, the full-rank target-task update was

$$\Delta W_{\text{FT}} = W_{\text{FT}} - W_0. \tag{88}$$

6.8.2 Post Hoc SVD of the Full-Rank Update

The first adaptation method computed a truncated SVD of the already-trained update:

$$\Delta W_{\text{FT},r} = \text{SVD}_r(\Delta W_{\text{FT}})\,, \tag{89}$$

and evaluated the effective matrix

$$W_0 + \Delta W_{\text{FT},r}. \tag{90}$$

This method therefore compressed a full-rank update after the update had already been learned.

6.8.3 Directly Learned LoRA Update

The second method reset the model to W_0, froze the dense base weights, and directly learned

$$\Delta W_{\text{LoRA}} = BA. \tag{91}$$

The implementation used the scaled form

$$W_{\text{effective}} = W_0 + \frac{\alpha}{r} BA, \tag{92}$$

with $\alpha = r$, so the scaling factor was one. The A factors were initialized from a zero-mean Gaussian distribution with standard deviation 0.01, while the B factors were initialized to zero. Consequently, every LoRA model exactly reproduced the source model before adaptation training began.

The equal-rank adaptation sweep used

$$r \in \{1, 2, 4, 8, 16, 32, 64, 96, 128\}. \tag{93}$$

Energy-matched update ranks were also evaluated:

$$(r_1, r_2) = (65, 55), \qquad (90, 70), \qquad (148, 95). \tag{94}$$

A validation-only pilot compared LoRA learning rates 10^{-3} and 3×10^{-3} at rank 16. The smaller rate achieved the lower validation loss and was therefore used for every LoRA rank. All LoRA models received the same 15-epoch budget and the same deterministic training-data order.

The two low-rank adaptation methods used the same number of task-specific parameters at a given rank pair:

$$N_{\text{adapter}} = 1040 r_1 + 384 r_2. \tag{95}$$

However, their training costs differed. The post hoc SVD method first required training the 233,472-parameter layer-matched full-rank update and only then compressed it. LoRA directly trained only the low-rank factors.

6.9 Software Environment and Random Seeds

Table 2 records the software environment used for the final run.

The value 42 was used for the Python, NumPy, PyTorch, dataset-splitting, DataLoader-shuffling, and LoRA-initialization seeds. The number of DataLoader workers was fixed at zero. Deterministic PyTorch algorithms were requested, cuDNN benchmarking was disabled, and the training loaders were recreated with the same shuffle seed for each adaptation method.

All results were produced from one deterministic execution rather than from repeated independent seeds. Accordingly, the experiment provides an exact reproducible comparison for this run but does not estimate variance, confidence intervals, or statistical significance.

Table 2: Software and hardware environment for the final experiment.

Component	Version or setting
Python	3.9.6
PyTorch	2.5.1
torchvision	0.20.1
NumPy	2.0.2
pandas	2.2.3
Operating system	macOS 26.5.2, ARM64
Compute device	CPU
CUDA	Not available
Training precision	`float32`
SVD and reconstruction precision	`float64`

7 Results

7.1 Baseline Performance

The dense source model selected at epoch 9 achieved a validation accuracy of 89.22% and a test accuracy of 88.38%. Its test cross-entropy loss was 0.3260. Table 3 summarizes the baseline results.

Table 3: Performance of the dense source model.

Model	Best epoch	Validation loss	Validation accuracy	Test loss	Test accuracy	Parameters
Dense source baseline	9	0.2995	89.22%	0.3260	88.38%	235,146

These values were reproduced after reloading the saved checkpoint, confirming that the stored baseline and the evaluated baseline were identical.

7.2 Singular-Value Spectra

Both selected weight matrices had gradually decaying singular-value spectra. The first singular values were approximately

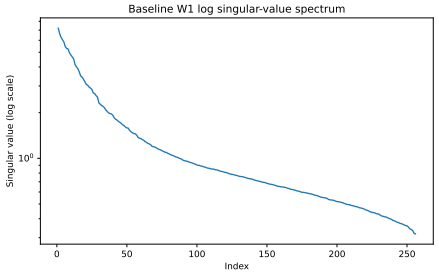
$$\sigma_1(W_1) = 7.208, \qquad \sigma_1(W_2) = 3.442, \tag{96}$$

while the final singular values were still nonzero:

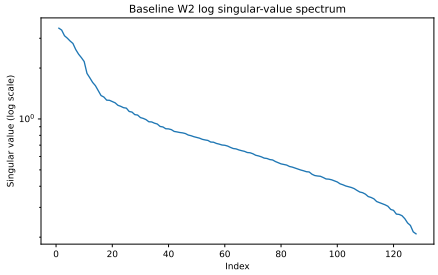
$$\sigma_{256}(W_1) = 0.318, \qquad \sigma_{128}(W_2) = 0.209. \tag{97}$$

Thus, neither matrix was exactly low rank. The spectra declined steeply over the first several directions and then entered a long, smooth tail rather than exhibiting a sharp cutoff. This suggests substantial approximate low-rank structure, but not a single unambiguous "true rank."

The ranks required to retain the specified energy levels are shown in Table 4.

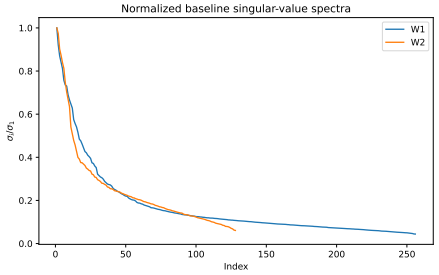


(a) Log-scale spectrum of W_1.

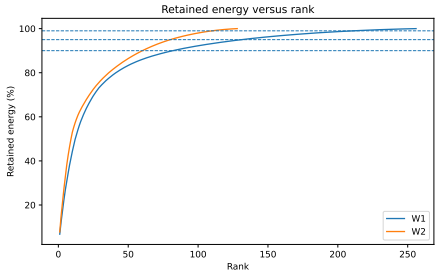


(b) Log-scale spectrum of W_2.

Figure 1: Singular-value spectra of the two selected source-model weight matrices. Both matrices have nonzero tails and therefore are not exactly low rank.



(a) Normalized singular values.



(b) Retained energy as a function of rank.

Figure 2: Comparison of spectral decay and cumulative retained energy for W_1 and W_2.

Table 4: Minimum ranks required to retain specified fractions of singular-value energy.

Layer	Energy target	Minimum rank	Fraction of maximum rank	Absolute error	Relative error
W_1	90%	82	32.0%	8.7558	0.3160
W_1	95%	132	51.6%	6.1522	0.2221
W_1	99%	212	82.8%	2.7276	0.0985
W_2	90%	61	47.7%	3.8442	0.3119
W_2	95%	81	63.3%	2.7082	0.2197
W_2	99%	110	85.9%	1.2285	0.0997

At the same absolute rank, W_2 generally retained slightly more energy. For example, at rank 32, W_1 retained 75.13% of its energy, whereas W_2 retained 77.57%. At rank 64, the corresponding values were 86.93% and 91.16%. Nevertheless, W_1 is much larger and dominates the dense model's parameter count, so reducing its rank creates larger whole-model storage savings. Consequently, whether one layer is "more compressible" depends on whether compressibility is measured by absolute rank, fraction of maximum rank, retained energy, or whole-model parameter reduction.

7.3 Numerical Verification of the Eckart–Young Theorem

The directly computed reconstruction error agreed with the theoretical tail-singular-value expression to floating-point precision. Across the rank sweep, the largest absolute discrepancy was approximately

$$9.95 \times 10^{-14} \quad \text{for } W_1, \qquad 3.34 \times 10^{-14} \quad \text{for } W_2. \tag{98}$$

At full rank, the small nonzero direct residuals were also on the order of 10^{-14}.

Therefore, the experiment numerically confirmed

$$\|W - W_r\|_F \approx \sqrt{\sum_{i=r+1}^{p} \sigma_i^2}. \tag{99}$$

It also confirmed the identity

$$\left(\frac{\|W - W_r\|_F}{\|W\|_F}\right)^2 + E_r = 1 \tag{100}$$

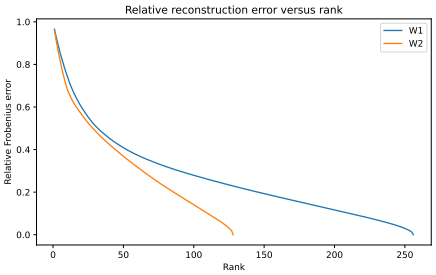
up to numerical precision. This connects the theoretical Eckart–Young result directly to the matrix approximations used in the classifier.

7.4 Classification Accuracy under SVD Compression

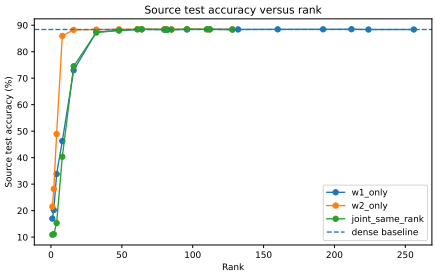
Representative compression results are shown in Table 5. For a single-layer experiment, the relative-error column refers to the compressed layer. For a joint experiment, it refers to the combined relative error defined in Equation (80).

Table 5: Representative source-model compression results. Accuracy drops are measured relative to the 88.38% dense baseline.

Configuration	r_1	r_2	Relative error	Test accuracy	Accuracy drop	Parameters	Parameter fraction	Compression factor
Dense baseline	–	–	0.0000	88.38%	0.00 pp	235,146	100.00%	1.00×
W_1 only	32	–	0.4987	87.39%	0.99 pp	67,722	28.80%	3.47×
W_2 only	–	16	0.6053	88.25%	0.13 pp	208,522	88.68%	1.13×
Joint, same rank	16	16	0.6419	74.50%	13.88 pp	24,458	10.40%	9.61×
Joint, same rank	32	32	0.4946	87.25%	1.13 pp	47,242	20.09%	4.98×
Joint, same rank	48	48	0.4107	88.02%	0.36 pp	70,026	29.78%	3.36×
Joint, same rank	64	64	0.3517	88.49%	−0.11 pp	92,810	39.47%	2.53×
Joint, 90%-energy pair	82	61	0.3154	88.28%	0.10 pp	110,378	46.94%	2.13×
Joint, 95%-energy pair	132	81	0.2217	88.48%	−0.10 pp	170,058	72.32%	1.38×
Joint, 99%-energy pair	212	110	0.0987	88.45%	−0.07 pp	264,394	112.44%	0.89×

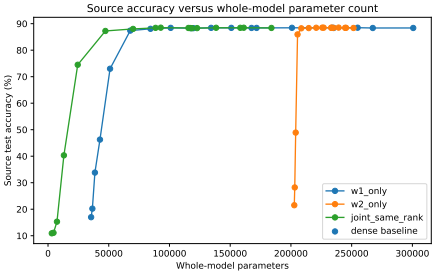


(a) Relative Frobenius error versus rank.

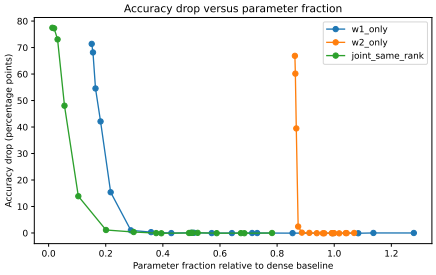


(b) Source-test accuracy versus rank.

Figure 3: Matrix-reconstruction error changes smoothly with rank, while classification accuracy exhibits a sharper transition at low and moderate ranks.



(a) Accuracy versus whole-model parameter count.



(b) Accuracy drop versus parameter fraction.

Figure 4: Tradeoff between source-task accuracy and whole-model compression.

Several patterns are visible. First, classification accuracy did not track Frobenius reconstruction error directly. The rank-16 approximation of W_2 had a relative error of 0.6053, meaning that more than 60% of the matrix's Frobenius norm remained in the residual, yet the model lost only 0.13 percentage points of test accuracy. In contrast, compressing W_1 to rank 16 reduced test accuracy to 73.00%, a loss of 15.38 percentage points. Thus, W_1 was much more accuracy-sensitive than W_2 at the same rank.

Second, accuracy showed a threshold-like recovery rather than the smooth behavior of reconstruction error. In the joint experiment, increasing the rank from 16 to 32 reduced the combined relative error from 0.6419 to 0.4946, but increased test accuracy from 74.50% to 87.25%. Increasing the rank again to 48 reduced the test-accuracy loss to only 0.36 percentage points. Reconstruction error decreased continuously with rank, whereas task accuracy changed sharply over a relatively narrow range.

Third, approximating both layers produced modest accumulated damage relative to compressing a single layer. At rank 32, the W_1-only model achieved 87.39%, the W_2-only model achieved 88.34%, and the joint model achieved 87.25%. The additional reduction was small but consistent with the fact that the second layer receives an already perturbed representation from the first layer.

Finally, several compressed configurations produced accuracies slightly above 88.38%. These differences were at most a few tenths of a percentage point and came from a single deterministic run. They should not be interpreted as evidence that SVD compression improved generalization. A more appropriate conclusion is that these configurations preserved baseline performance to the resolution of the experiment.

These findings demonstrate the distinction between the matrix and task objectives. Truncated SVD is guaranteed to minimize

$$\|W - \widehat{W}\|_F \tag{101}$$

among all rank-r matrices, but it does not identify which singular directions are most important for the classifier's decision boundaries.

7.5 Parameter-Compression Results

Validation performance was used to select representative efficient configurations without consulting test accuracy. Among all compressed models, the smallest model whose validation accuracy was within one percentage point of the dense validation accuracy was the joint rank-48 model. It contained 70,026 parameters, or 29.78% of the dense model, corresponding to a 70.22% parameter reduction and a 3.36× compression factor. Its test accuracy was 88.02%, only 0.36 percentage points below the baseline.

Using a two-percentage-point validation tolerance selected the joint rank-32 model. It contained 47,242 parameters, or 20.09% of the original model. This represents a 79.91% parameter reduction and a 4.98× compression factor, with a test-accuracy drop of 1.13 percentage points.

The layer-specific 90%-energy configuration $(r_1, r_2) = (82, 61)$ retained 90.01% of the energy of W_1 and 90.27% of the energy of W_2. It reduced the model to 46.94% of its original parameter count while losing only 0.10 percentage points of test accuracy.

In contrast, retaining 99% of the energy was not an effective compression point. The $(212, 110)$ factorization contained 264,394 parameters, which was 12.44% more than the dense model. Similarly, the exact factorized representation $(256, 128)$ reproduced the original model numerically but contained 317,066 parameters, a 34.84% increase. These cases confirm that a low-rank representation should only be called a compression when the factorized parameter count is actually smaller than the dense parameter count.

These results quantify parameter reduction, but they do not establish proportional reductions in runtime or wall-clock latency. The experiment did not benchmark inference speed, memory transfers, or hardware-specific matrix-multiplication efficiency.

7.6 Target-Task Reference Performance

The $+20°$ rotation created a substantial domain shift. Without adaptation, the source model's target-test accuracy fell from 88.38% to 45.77%, a decrease of 42.61 percentage points. Table 6 compares zero-shot performance with the two full-rank adaptation references.

Table 6: Target-task reference performance.

Adaptation method	Trainable parameters	Best epoch	Target-test loss	Target-test accuracy	Source-test accuracy
Source model, zero shot	0	–	1.9623	45.77%	88.38%
Unrestricted full-model fine-tuning	235,146	12	0.3287	88.26%	55.70%
Selected-layer full-rank adaptation	233,472	15	0.3281	88.77%	59.39%

Figure 5: Target-task performance of the zero-shot source model and the two full-rank adaptation references.

Both full-rank adaptation methods recovered most of the target-domain per-

formance. The selected-layer reference achieved 88.77%, which was 0.51 percentage points above unrestricted fine-tuning in this run. Because only one seed was evaluated, this small difference should not be interpreted as evidence that freezing the other parameters is generally superior. The selected-layer result is important primarily because it provides a fair reference update for the SVD-versus-LoRA comparison.

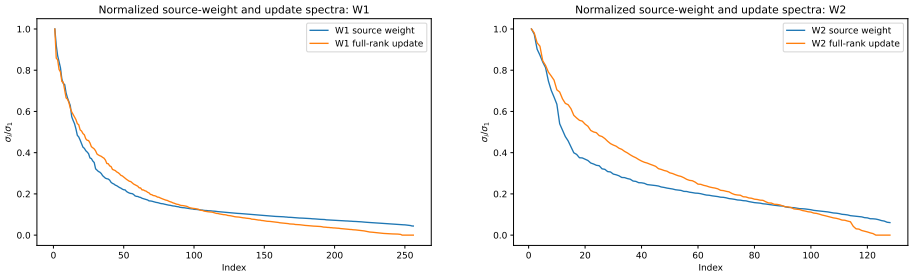
The source-test accuracies of 55.70% and 59.39% also show that adaptation to the rotated domain substantially changed the original source behavior. The target update therefore represented a meaningful domain adaptation rather than a negligible perturbation.

7.7 Singular Values of the Full-Rank Update

The full-rank selected-layer updates were more spectrally concentrated than the original source weights. The update ranks required for the energy thresholds are reported in Table 7.

Table 7: Minimum ranks required to retain the specified fractions of the full-rank update energy.

Layer update	90% **energy**	95% **energy**	99% **energy**
ΔW_1	65	90	148
ΔW_2	55	70	95



(a) W_1 and ΔW_1.

(b) W_2 and ΔW_2.

Figure 6: Comparison of the source-weight spectra with the spectra of the selected-layer full-rank updates.

For comparison, the original W_1 required ranks 82, 132, and 212, while the original W_2 required ranks 61, 81, and 110. This indicates that the learned target-task changes were more compressible in Frobenius energy than the complete source matrices.

However, the updates were not small in Frobenius norm. The ratios

$$\frac{\|\Delta W_1\|_F}{\|W_{1,0}\|_F} = 1.109, \qquad \frac{\|\Delta W_2\|_F}{\|W_{2,0}\|_F} = 1.070 \tag{102}$$

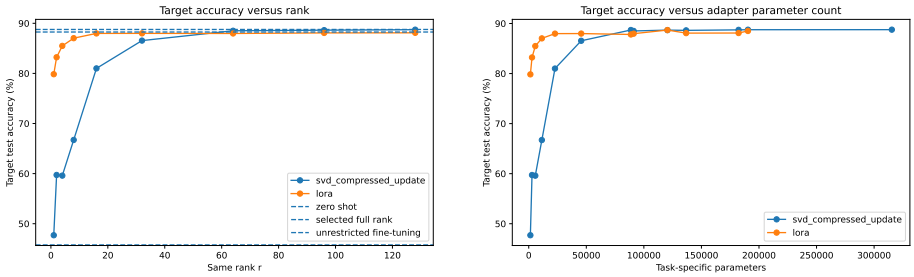
show that the full-rank update norms were comparable to, and slightly larger than, the corresponding base-weight norms.

7.8 Post Hoc SVD versus Directly Learned LoRA

Table 8 compares rank-matched post hoc SVD updates and directly learned LoRA updates. Both methods used the same number of stored or trainable low-rank parameters for each rank pair.

Table 8: Rank-matched comparison of post hoc SVD compression and directly learned LoRA adaptation. The accuracy difference is LoRA accuracy minus SVD accuracy.

r_1	r_2	SVD target accuracy	LoRA target accuracy	LoRA − SVD	Adapter parameters
1	1	47.70%	79.84%	+32.14 pp	1,424
4	4	59.60%	85.49%	+25.89 pp	5,696
8	8	66.72%	87.03%	+20.31 pp	11,392
16	16	81.00%	87.98%	+6.98 pp	22,784
32	32	86.55%	87.99%	+1.44 pp	45,568
64	64	88.51%	87.97%	−0.54 pp	91,136
90	70	88.68%	88.69%	+0.01 pp	120,480
128	128	88.73%	88.10%	−0.63 pp	182,272
148	95	88.76%	88.51%	−0.25 pp	190,400



(a) Target accuracy versus rank. (b) Target accuracy versus adapter size.

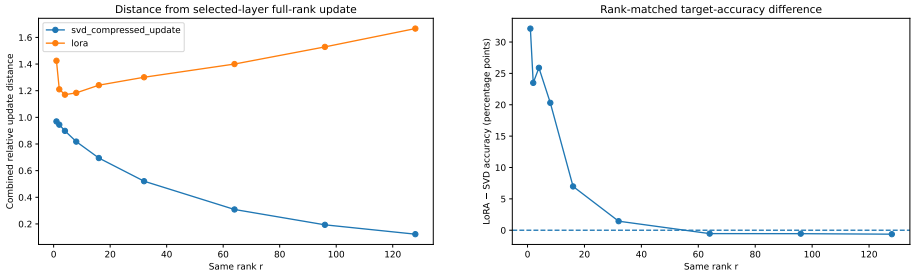
Figure 7: Target-task accuracy of post hoc SVD and LoRA under rank-matched and parameter-matched comparisons.

LoRA was substantially more effective at very small ranks. At rank 1, LoRA achieved 79.84% target accuracy, while the post hoc SVD update achieved only 47.70%. At rank 8, LoRA remained 20.31 percentage points ahead. Rank-16 LoRA reached 87.98%, only 0.79 percentage points below the selected-layer full-rank reference while training 22,784 parameters, or approximately 9.76% of the 233,472 full-rank selected-layer parameters.

The gap narrowed at larger ranks. At rank 32, LoRA retained a 1.44-percentage-point advantage. At rank 64, post hoc SVD became 0.54 percentage points better, and the two methods were essentially tied at the 95%-energy pair $(90, 70)$. The best LoRA result was 88.69% at $(90, 70)$, only 0.08 percentage points below the selected-layer full-rank reference. The 99%-energy SVD update reached 88.76%, only 0.01 percentage points below the full-rank reference.

The update-distance results further illustrate the difference between the methods. For an approximation $(\widehat{\Delta W}_1, \widehat{\Delta W}_2)$, the combined relative distance from the selected full-rank update was

$$d = \frac{\sqrt{\|\Delta W_1 - \widehat{\Delta W}_1\|_F^2 + \|\Delta W_2 - \widehat{\Delta W}_2\|_F^2}}{\sqrt{\|\Delta W_1\|_F^2 + \|\Delta W_2\|_F^2}}. \tag{103}$$



(a) Distance from the full-rank update. (b) LoRA accuracy minus SVD accuracy.

Figure 8: The SVD update becomes progressively closer to the selected full-rank update, while LoRA can achieve high task accuracy without approximating that particular update closely.

For post hoc SVD, the distance in Equation (103) decreased from 0.969 at rank 1 to 0.100 at the 99%-energy pair. This is expected because truncated SVD is explicitly constructed to approximate the full-rank update in Frobenius norm.

In contrast, the LoRA distances remained greater than one for every evaluated configuration. For example, the distances were 1.425 at rank 1, 1.241 at rank 16, 1.301 at rank 32, and 1.510 at the 95%-energy pair. Nevertheless, the LoRA models achieved target accuracies close to the full-rank reference. LoRA therefore did not recover a close matrix approximation to the particular update learned by full-rank fine-tuning. Instead, it found a different low-rank update that performed well on the same task.

This is the central empirical distinction between the two approaches. Post hoc SVD optimizes matrix reconstruction:

$$\min_{\operatorname{rank}(\widehat{\Delta W}) \le r} \left\| \Delta W_{\mathrm{FT}} - \widehat{\Delta W} \right\|_F. \tag{104}$$

LoRA directly optimizes target classification loss over a low-rank parameterization. At small ranks, a task-optimized update can therefore outperform the best Frobenius approximation of one particular full-rank solution. At larger ranks, the SVD approximation retains enough of the full-rank update to match or slightly exceed the LoRA result from this training configuration.

The parameter comparison also requires care. A rank-(r_1, r_2) SVD adapter and a rank-matched LoRA adapter store the same number of low-rank values. However, producing the SVD adapter first required training the full 233,472-parameter selected-layer model. LoRA trained only the low-rank factors. Thus, SVD reduced task-specific storage after training, while LoRA reduced the number of trainable adaptation parameters during training.

8 Discussion

The central question of this project was whether the best low-rank approximation of a neural-network weight matrix in the Frobenius norm also preserves the classification behavior of the network. The numerical results show that these two objectives are related, but they are not equivalent. The singular-value spectra reveal substantial concentration of matrix energy in a smaller number of directions, and the Eckart–Young theorem accurately predicts the reconstruction error obtained by discarding the remaining directions. However, the classification experiments show that the effect of this approximation depends not only on the size of the discarded singular values, but also on the layer being compressed, the data distribution, and the role of the discarded directions in the network's predictions.

8.1 Interpretation of the Singular-Value Spectra

The singular values of both W_1 and W_2 decreased gradually rather than dropping abruptly to zero. The smallest singular values of both matrices were nonzero, so neither trained weight matrix was exactly low rank. There was also no single sharp elbow in either spectrum that would identify an obvious intrinsic rank. Therefore, it would be inaccurate to describe either matrix as literally low rank.

Nevertheless, the spectra do show meaningful approximate low-rank structure. A relatively small initial portion of each spectrum accounted for a large fraction of the matrix's squared Frobenius norm. For W_1, ranks 82, 132, and 212 retained approximately 90%, 95%, and 99% of the singular-value energy. For W_2, the corresponding ranks were 61, 81, and 110. Thus, both layers could discard a substantial number of singular directions while retaining most of their matrix energy.

These energy thresholds should be interpreted as descriptive operating points rather than as uniquely correct compression ranks. In particular, retaining 99% of the energy was unnecessarily conservative for this architecture. The 99%-energy joint factorization contained more parameters than the original dense

model, even though it reconstructed the matrices very accurately. By contrast, lower-energy configurations often preserved essentially all classification accuracy while producing substantially greater parameter reduction. This demonstrates that an energy threshold alone is not sufficient for selecting a useful neural-network compression rank.

The spectra of the full fine-tuning updates were more concentrated than the spectra of the original source weights. The update ΔW_1 required ranks 65, 90, and 148 to retain 90%, 95%, and 99% of its energy, while ΔW_2 required ranks 55, 70, and 95. This suggests that, in this experiment, the change needed to adapt the model to rotated Fashion–MNIST occupied fewer dominant directions than the complete source matrices. However, this observation concerns one learned update from one optimization run. It does not imply that all fine-tuning updates, datasets, or architectures will have similarly concentrated spectra.

8.2 Where Classification Accuracy Begins to Decline

The accuracy results did not show a single universal critical rank. Instead, the practical transition depended strongly on which layer was compressed.

For W_1-only compression, rank 32 still produced 87.39% test accuracy, a decrease of only 0.99 percentage points from the 88.38% baseline. Reducing the rank further to 16, however, lowered accuracy to 73.00%, a decrease of 15.38 percentage points. The important transition for the first layer therefore occurred between ranks 16 and 32. Below this range, the discarded directions removed information that the remainder of the network could not recover.

The second layer was considerably less sensitive. Compressing W_2 to rank 16 resulted in 88.25% accuracy, only 0.13 percentage points below the baseline, even though the relative Frobenius reconstruction error was 0.6053. At rank 8, accuracy fell to 85.95%, producing a more visible but still moderate loss of 2.43 percentage points. At rank 4, accuracy fell sharply to 48.91%. Thus, W_2 remained highly tolerant down to rank 16, began to show a meaningful decline near rank 8, and deteriorated severely below that point.

The joint experiment displayed a transition similar to that of W_1, which is consistent with the first layer being the more accuracy-sensitive component. Joint rank 48 achieved 88.02% test accuracy, only 0.36 percentage points below the baseline, while using 29.78% of the original model parameters. Joint rank 32 achieved 87.25%, a decrease of 1.13 percentage points. At joint rank 16, however, accuracy fell to 74.50%, a decrease of 13.88 percentage points. The main transition again occurred between ranks 16 and 32.

These findings suggest two useful practical operating points. Joint rank 48 is the more conservative choice: it removes 70.22% of the model parameters while nearly preserving baseline accuracy. Joint rank 32 is a more aggressive choice: it removes 79.91% of the parameters at the cost of approximately 1.13 percentage points of test accuracy. Ranks below 32 produce larger savings, but the accuracy cost rises rapidly.

The small nonmonotonic changes at higher ranks should not be overinterpreted. Several compressed configurations performed a few hundredths or tenths

of a percentage point above the baseline. Such results are possible because a perturbation can move a small number of test examples across the correct decision boundary. However, one deterministic model and one test set do not establish that compression systematically improves generalization. The appropriate conclusion is that these ranks preserved baseline-level performance within the resolution of the experiment.

8.3 Why Frobenius Reconstruction Error and Accuracy Differ

The difference between reconstruction error and classification accuracy is explained by the fact that they measure different objectives. Truncated SVD solves the matrix problem

$$\min_{\operatorname{rank}(\widehat{W})\leq r} \|W - \widehat{W}\|_F. \tag{105}$$

This objective treats the approximation as a problem involving all entries and all directions of the matrix. It is independent of the dataset, labels, activation patterns, and classification loss. A discarded singular direction contributes to the Frobenius error according to its singular value, regardless of whether that direction is frequently used by the input data or important for distinguishing classes.

Classification accuracy instead depends on the composed network function. For an input x, replacing W with W_r matters only through the effect of the perturbation on the activations that the network actually encounters. Several factors can weaken or amplify that effect.

First, Fashion–MNIST images occupy only a restricted part of the full 784-dimensional input space. A singular direction can contribute noticeably to the matrix norm while being weakly activated by the data. Removing such a direction increases Frobenius error without substantially changing predictions.

Second, the ReLU nonlinearities make the effect of a weight perturbation input-dependent. A change in a hidden preactivation may have no effect if the unit remains on the same side of zero, but a smaller change may matter greatly if it changes whether a hidden unit is active. Consequently, two approximations with similar matrix errors can affect the network differently.

Third, accuracy depends on classification margins. If the correct class logit is much larger than every competing logit, a substantial perturbation may leave the predicted class unchanged. If two logits are nearly tied, a much smaller perturbation may change the prediction. Accuracy is therefore a discrete and threshold-based quantity, whereas Frobenius error varies continuously.

Fourth, later layers can suppress, compensate for, or amplify the perturbation introduced by an earlier layer. The network does not use every hidden direction equally. Some directions may contribute strongly to the norm of a weight matrix but have little effect on the final logits, while other directions may be essential for particular class distinctions.

The contrast between the two layers illustrates this difference. At rank 16, the relative errors of W_1 and W_2 were both large: approximately 0.649 and 0.605, respectively. Nevertheless, compressing W_1 caused a 15.38-percentage-point accuracy loss, whereas compressing W_2 caused only a 0.13-percentage-point loss. Frobenius error alone therefore could not predict the task sensitivity of the two layers.

The joint experiment also shows that layerwise approximation errors do not translate into accuracy through a simple additive rule. When the first layer is approximated, the second layer receives a changed hidden representation. Approximating the second layer then acts on this already modified input. The resulting interaction depends on the nonlinear network function rather than only on the two individual matrix errors.

This does not weaken the Eckart–Young theorem. The theorem answers its intended question exactly: truncated SVD is the best rank-r approximation under the Frobenius norm. The experiment instead shows that the mathematically optimal solution to this matrix objective is not automatically optimal for a downstream classification objective.

8.4 Which Layer Is Most Compressible?

There is no single answer to which layer was "most compressible," because the answer depends on the criterion being used.

From the perspective of classification sensitivity, W_2 was clearly more compressible. It could be reduced to rank 16, one eighth of its maximum rank, with almost no loss of accuracy. At the same rank, W_1 suffered a large performance decrease. This indicates that the network relied more critically on the directions retained by the first layer.

From the perspective of parameter reduction, however, W_1 is the more important compression target. It contains 200,704 weights, or approximately 85% of all parameters in the dense model. Compressing W_2 alone cannot produce a similarly large whole-model reduction because W_1 remains unchanged. For example, rank-16 compression of W_2 preserved accuracy but still left the whole model with 208,522 parameters, or 88.68% of the original total.

From the perspective of retained energy relative to maximum possible rank, W_1 also appears somewhat more concentrated. It reached 90% energy at rank 82, which is approximately 32.0% of its maximum rank, whereas W_2 reached 90% energy at rank 61, approximately 47.7% of its maximum rank. At the same absolute rank, however, W_2 generally retained slightly more energy. These apparently different conclusions arise because the matrices have different dimensions and maximum ranks.

The most accurate interpretation is therefore that W_2 was more tolerant of aggressive low-rank approximation, while W_1 offered greater potential parameter savings but required a larger rank to preserve the network's predictions. A strong compression strategy should account for both facts rather than assigning the same rank to every layer automatically.

The energy-matched joint configurations support this conclusion. Assigning ranks $(82, 61)$ allowed the two layers to retain approximately 90% of their respective energies and reduced the whole model to 46.94% of its original parameter count, with only a 0.10-percentage-point accuracy decrease. However, the equal-rank 48 model achieved greater parameter reduction with only a slightly larger accuracy decrease. This shows that retained energy provides a principled way to interpret rank, but it is not necessarily the best criterion for choosing a task-efficient rank allocation.

More generally, layer-specific rank selection is preferable to assuming that one rank is optimal for the entire network. The first and second layers have different dimensions, parameter costs, singular-value distributions, and sensitivity to perturbation. A more advanced compression method could select r_1 and r_2 jointly by optimizing validation accuracy subject to a parameter budget.

8.5 Interpretation of the LoRA Comparison

The LoRA experiment provides an especially clear example of the distinction between a matrix objective and a task objective. At very small ranks, directly learned LoRA updates substantially outperformed truncated-SVD approximations of the full-rank fine-tuning update. At rank 1, LoRA achieved 79.84% target accuracy, compared with 47.70% for the SVD-compressed update. At rank 8, the corresponding accuracies were 87.03% and 66.72%. At rank 16, LoRA achieved 87.98%, only 0.79 percentage points below the selected-layer full-rank reference, while training fewer than 10% as many selected-layer parameters.

This low-rank advantage is consistent with the objectives of the two methods. Post hoc SVD was constrained to approximate the particular update found by full-rank adaptation:

$$\Delta W_{\mathrm{FT}} = W_{\mathrm{FT}} - W_0. \tag{106}$$

It selected the rank-r matrix closest to that update in the Frobenius norm. LoRA was not required to approximate this particular full-rank update. It directly searched within the set of low-rank updates for one that reduced target-task loss.

The update-distance measurements reinforce this interpretation. As rank increased, the SVD-compressed update became progressively closer to the selected full-rank update. The LoRA updates remained relatively far from that update in the Frobenius norm, yet they achieved similar target accuracy. This means that a low-rank update did not need to reproduce the full-rank optimization path in order to produce a similar classifier. Multiple different weight updates can lead to similar predictions, especially in an overparameterized neural network.

The advantage of LoRA was concentrated at low ranks. At rank 32, LoRA remained 1.44 percentage points ahead of post hoc SVD. At rank 64, however, SVD was 0.54 percentage points ahead. At the energy-matched pair $(90, 70)$, the two methods were almost identical, achieving 88.68% and 88.69%. Thus, the results do not support the claim that LoRA is always superior. They instead show that direct task optimization was particularly valuable when the update

rank was severely constrained. Once the SVD approximation had enough rank to preserve most of the full-rank update, the difference largely disappeared.

The parameter interpretation must also be stated carefully. In the rank-matched adaptation comparison, an SVD update and a LoRA update contained the same number of task-specific low-rank parameters. However, the SVD method required first training the complete selected-layer full-rank update and then compressing it. LoRA trained only the low-rank factors from the beginning. LoRA therefore reduced the number of trainable adaptation parameters, whereas post hoc SVD reduced the storage required for an update after full-rank training had already occurred.

LoRA did not compress the underlying source model. The frozen base matrices W_0 were still required to evaluate

$$W_{\text{effective}} = W_0 + BA. \tag{107}$$

Therefore, the LoRA result should be interpreted as parameter-efficient adaptation, not as evidence that the complete deployed model had been reduced to the size of the adapter. This differs from the source-model SVD experiment, in which a dense matrix was actually replaced by smaller factors when the selected rank satisfied the parameter-compression condition.

The comparison establishes that, for this specific MLP, target transformation, training configuration, and seed, directly optimizing a low-rank update was much more effective than compressing one particular full-rank update at very small ranks. It does not establish that LoRA universally outperforms post hoc SVD, that the selected LoRA hyperparameters are optimal, or that the same ordering will hold for other architectures and downstream tasks.

8.6 Relationship to the Eckart–Young Theorem

The theoretical and computational portions of the project address two connected but distinct questions. The Eckart–Young theorem guarantees that

$$W_r = \sum_{i=1}^{r} \sigma_i u_i v_i^\top \tag{108}$$

minimizes the Frobenius reconstruction error among all matrices of rank at most r. The numerical experiment verified this guarantee to floating-point precision by confirming that the direct error matched

$$\sqrt{\sum_{i=r+1}^{p} \sigma_i^2}. \tag{109}$$

The theorem therefore provides an exact mathematical foundation for the compression procedure and for interpreting the singular-value tail.

What the theorem does not determine is how the approximation changes the function computed by the network. It does not include the distribution of

Fashion–MNIST inputs, the ReLU activations, later layers, classification margins, or the cross-entropy objective. Those quantities enter only in the empirical evaluation.

The overall conclusion is therefore not that Eckart–Young fails for neural networks. Rather, it succeeds exactly at the matrix problem it solves, while neural-network compression introduces an additional task-dependent problem. A matrix can be well approximated in the Frobenius norm without preserving every important prediction, and a model can preserve its predictions even when the matrix reconstruction error is substantial. The useful compression ranks are determined by the interaction of both considerations.

9 Limitations

The experiment was designed to provide a controlled and reproducible connection between low-rank matrix approximation and neural-network behavior. Its conclusions must nevertheless be restricted to the conditions that were actually tested.

9.1 Limited Dataset and Model Scope

The source experiment used one Fashion–MNIST classifier with a single multilayer-perceptron architecture,

$$784 \longrightarrow 256 \longrightarrow 128 \longrightarrow 10. \tag{110}$$

Fashion–MNIST is a relatively small grayscale image dataset, and a fully connected network does not use the convolutional structure that is common in modern image classifiers. The observed spectra, layer sensitivities, and useful rank ranges may differ for convolutional networks, residual networks, transformers, or substantially larger models.

Only one hidden-layer width configuration was evaluated. Changing the number of hidden units, regularization, activation function, optimizer, or training duration could change both the learned weight matrices and their singular-value distributions. The experiment therefore demonstrates what occurred for this trained model, not an architecture-independent property of neural networks.

9.2 Single Deterministic Seed

All experiments used the single seed 42. Fixing the seed made the run reproducible and ensured that the compared methods used consistent data splits, initialization procedures, and training order. However, a single deterministic run does not estimate variability across random initializations or minibatch orders.

No standard deviations, confidence intervals, or statistical significance tests were calculated. Differences of only a few tenths of a percentage point—such as

compressed configurations that slightly exceeded the baseline, the 0.51-percentage-point difference between the two full-rank target references, or the small LoRA–SVD differences at high ranks—should therefore be treated as descriptive outcomes of this run rather than evidence of a reliable general advantage.

A stronger experiment would repeat baseline training, compression, full-rank adaptation, and LoRA training across several independent seeds and report means and variability.

9.3 Restricted Training and Hyperparameter Search

The source model used one optimizer configuration and a maximum budget of 20 epochs. The adaptation methods used a fixed 15-epoch budget. These choices produced a suitable model for the project, but they were not established through an exhaustive search.

The LoRA learning-rate pilot compared only 10^{-3} and 3×10^{-3}, and it was conducted only at rank 16. The selected learning rate was then reused for every rank. Other LoRA choices—including rank-specific learning rates, weight decay, initialization scale, the value of α, training duration, or learning-rate schedules—could alter the comparison.

Consequently, the experiment compares the implemented versions of LoRA and post hoc SVD, not the best possible version of each method after extensive tuning. In particular, a weaker LoRA result at a high rank may reflect optimization choices rather than an inherent disadvantage of LoRA.

9.4 Controlled but Narrow Adaptation Task

The target domain was created by applying a fixed $+20^\circ$ rotation to Fashion–MNIST images. This produced a clear and reproducible distribution shift while preserving the original labels. However, it is an artificial transformation of the same dataset rather than a separate real-world downstream task.

Rotation introduces specific interpolation and boundary effects, and the observed adaptation structure may be particular to this transformation. Different rotation angles, corruptions, style changes, class distributions, or entirely different datasets could produce updates with different singular-value spectra and different relative performance between LoRA and SVD.

The experiment therefore supports conclusions about this controlled source-to-target shift only. It does not establish how the methods compare across domain-adaptation problems in general.

9.5 Limited Layer Coverage

The source-compression and adaptation experiments modified only W_1 and W_2. The output matrix W_3, all bias vectors, and other possible architectural components were excluded. This was reasonable because W_3 had maximum rank 10, making it less informative for a broad sweep, but it means that the experiment does not describe the sensitivity of every model parameter.

Most joint experiments also used the same rank in both selected layers. Only the energy-matched configurations tested unequal ranks. Since the two layers showed different spectral and task sensitivities, a broader search over pairs (r_1, r_2) might identify better accuracy–compression tradeoffs.

The conclusion that W_2 is more accuracy-tolerant than W_1 is therefore specific to these two matrices in this trained network. It should not be interpreted as a general statement that later neural-network layers are always more compressible than earlier layers.

9.6 No Retraining after Source Compression

The truncated-SVD source models were evaluated without retraining or post-compression fine-tuning. This was an intentional design choice: it isolated the immediate consequence of replacing a trained weight matrix by its Eckart–Young approximation.

However, the resulting accuracy curves do not represent the best performance achievable by a network with the same low-rank architecture. Fine-tuning the factors after compression could allow the model to recover some of the lost accuracy. Training the network with low-rank factors from the beginning could also produce a different solution.

The severe accuracy losses at very small ranks therefore show that direct post hoc replacement failed at those ranks. They do not prove that no rank-r neural network could achieve high Fashion–MNIST accuracy.

9.7 Dependence on One Full-Rank Adaptation Solution

The post hoc SVD adaptation method approximated one particular selected-layer full-rank update produced by gradient-based training. Neural-network optimization is not guaranteed to produce a unique update, and different seeds or hyperparameters could reach different parameter values with similar target accuracy.

As a result, the Frobenius distance from this particular update is not an absolute measure of task quality. The observation that LoRA remained far from the full-rank update while preserving target accuracy is informative, but it should not be interpreted as showing that LoRA is far from every possible full-rank solution. It instead shows that the full-rank update used in this run was not the only parameter change capable of producing strong target-task performance.

Similarly, truncated SVD was optimal for approximating that update under the Frobenius norm. It was not guaranteed to be the rank-r update that maximized target accuracy.

9.8 Limited Evaluation Metrics

The experiment reported cross-entropy loss, overall classification accuracy, Frobenius reconstruction error, retained energy, parameter count, and update dis-

tance. These metrics address the main research question, but they do not provide a complete evaluation of model behavior.

The analysis did not report per-class accuracy, confusion matrices, calibration, robustness to additional perturbations, worst-case performance, or changes in prediction confidence. A compressed model could preserve overall accuracy while disproportionately harming particular clothing classes.

Only the Frobenius norm was studied as a reconstruction metric. Other matrix norms, such as the spectral norm, emphasize different properties of the approximation and could relate differently to changes in the network function. The experiment therefore supports conclusions specifically about Frobenius-optimal approximation.

9.9 Parameter Count Is Not the Same as Runtime

The compression calculations establish reductions in the number of stored matrix parameters. They do not establish equivalent reductions in inference time, training time, energy consumption, or memory usage on actual hardware.

A factorized layer replaces one dense multiplication with two smaller multiplications. At sufficiently low ranks, this can reduce arithmetic cost, but the realized speed depends on batch size, implementation, memory movement, framework overhead, and hardware efficiency. Two smaller operations can sometimes be slower than one highly optimized dense operation.

No latency, throughput, floating-point-operation count, peak-memory, or energy benchmarks were collected. The report should therefore claim parameter reduction and task-specific trainable-parameter reduction only. It should not claim an inference-speed improvement.

For LoRA, the distinction is even more important. The adapter contains relatively few trainable parameters, but the frozen base model must still be stored and evaluated. LoRA therefore reduced adaptation cost in this experiment; it did not remove the storage requirement of the source model.

9.10 Scope of the Conclusions

The experiment numerically verifies the Eckart–Young reconstruction formula for the selected matrices and demonstrates several empirical relationships for one reproducible setting. It shows that moderate-rank compression can preserve Fashion–MNIST accuracy, that W_1 and W_2 have different task sensitivities, that Frobenius error does not directly determine classification accuracy, and that directly learned LoRA updates can be more effective than post hoc SVD updates at very small ranks in the chosen adaptation task.

It does not prove that the identified ranks are universally optimal, that the same layers will be most compressible in other models, that LoRA is generally superior to SVD, or that parameter reduction will produce proportional runtime improvement. Those questions would require experiments across additional architectures, datasets, transformations, random seeds, training procedures, and hardware environments.

10 Conclusion

This project investigated whether the mathematically optimal rank-r approximation of a trained neural-network weight matrix also preserves the classification behavior of the network. The results provide a qualified answer. Truncated SVD can preserve near-baseline classification accuracy over a useful range of moderate ranks while substantially reducing the parameter count. However, the Eckart–Young theorem's guarantee of optimality in the Frobenius norm does not, by itself, determine which ranks will preserve predictive performance. Matrix reconstruction and neural-network accuracy are connected, but they are different objectives.

The theoretical component established the mathematical foundation for the experiment. The singular value decomposition expresses a matrix as a sum of mutually orthogonal rank-one transformations ordered by their singular values. The Eckart–Young theorem then shows that retaining the first r components produces the best rank-r approximation under the Frobenius norm, with error

$$\|A - A_r\|_F = \left(\sum_{i=r+1}^{p} \sigma_i^2 \right)^{1/2}.$$

The computational experiment verified this formula numerically for both selected neural-network weight matrices. The largest discrepancies between the directly calculated reconstruction errors and the theoretical tail-singular-value errors were below 10^{-13}. The theorem therefore described the matrix approximations used in the experiment to floating-point precision.

The singular-value spectra of W_1 and W_2 showed meaningful approximate low-rank structure, but neither matrix was exactly low rank. Both spectra declined smoothly and retained nonzero singular-value tails, so there was no unique intrinsic rank at which the matrices could be truncated without loss. The classification experiments further showed that singular-value energy alone was not sufficient for selecting a useful rank. Retaining 99% of the energy produced highly accurate matrix reconstructions, but the corresponding joint factorization contained 264,394 parameters, which was more than the 235,146-parameter dense model. A factorization should therefore be called a compression only when its two factors actually contain fewer parameters than the matrix they replace.

The most useful accuracy–compression tradeoffs occurred at substantially lower ranks. Using validation performance to select a representative operating point, the joint rank-48 model contained 70,026 parameters, or 29.78% of the dense model. This corresponds to a 70.22% parameter reduction and a 3.36× compression factor, while its test accuracy of 88.02% was only 0.36 percentage points below the 88.38% baseline. The more aggressive joint rank-32 model contained 47,242 parameters, or 20.09% of the original model, and achieved 87.25% test accuracy. It therefore removed 79.91% of the parameters at the cost of a 1.13-percentage-point accuracy decrease.

Accuracy did not decrease smoothly with Frobenius reconstruction quality. Instead, it exhibited a threshold-like transition at low and moderate ranks. In the joint experiment, accuracy increased from 74.50% at rank 16 to 87.25% at rank 32, even though the reconstruction error changed continuously. The two selected layers also had markedly different task sensitivities. Compressing W_1 to rank 16 reduced test accuracy to 73.00%, whereas compressing W_2 to the same rank preserved 88.25% accuracy. Thus, W_2 tolerated a much more aggressive rank reduction, while W_1 offered greater potential parameter savings because it contained most of the model's weights.

These observations explain why Frobenius error cannot be used as a complete proxy for classification performance. The Frobenius norm measures the total matrix perturbation across all input directions without considering which directions occur frequently in the data or which directions are important for distinguishing classes. Classification behavior additionally depends on the Fashion–MNIST input distribution, the ReLU activation patterns, the margins between competing output logits, and the way later layers suppress or amplify perturbations. A discarded direction can contribute to matrix error while having little effect on the test inputs, whereas a direction with relatively little Frobenius energy can still be important for particular predictions.

The adaptation experiment reinforced the distinction between a matrix objective and a task objective. The $+20^\circ$ rotation reduced the zero-shot target accuracy to 45.77%, while selected-layer full-rank adaptation recovered 88.77%. At small ranks, directly learned LoRA updates were substantially more effective than post hoc truncated-SVD approximations of the full-rank update. At rank 16, LoRA achieved 87.98% target accuracy while training 22,784 low-rank parameters, approximately 9.76% of the selected-layer full-rank parameter count. The rank-16 post hoc SVD update achieved only 81.00%.

This advantage did not persist uniformly at larger ranks. At rank 64, the post hoc SVD update slightly exceeded the LoRA result, and the two methods were nearly identical at the energy-matched rank pair $(90, 70)$. The appropriate conclusion is therefore not that LoRA is universally superior to truncated SVD. Rather, directly optimizing a low-rank update for the target loss was particularly valuable when the permitted rank was very small. LoRA could identify a task-effective update that was far from the particular full-rank update in Frobenius distance, while post hoc SVD was restricted to reconstructing that full-rank solution as closely as possible. LoRA also remained an adaptation method rather than a compression of the complete base model, since the frozen source matrices were still required during inference.

The scope of these conclusions is limited by the use of one multilayer perceptron, one dataset, one deterministic seed, and one controlled rotation-based adaptation task. The source-compression models were not fine-tuned after factorization, and no runtime, latency, or hardware-specific memory measurements were collected. Future work could repeat the experiments across multiple seeds and architectures, optimize different ranks for different layers under a fixed parameter budget, fine-tune the SVD factors after compression, evaluate additional matrix norms and per-class metrics, and measure whether the theoretical

arithmetic reductions produce practical speedups.

The central conclusion is that the Eckart–Young theorem provides an exact and valuable mathematical foundation for low-rank neural-network approximation, but it solves only the matrix-reconstruction portion of the problem. Effective neural-network compression additionally requires task-level evaluation. Truncated SVD identifies the rank-r matrix that is closest to the original matrix, while the useful rank for a neural network depends on how the retained and discarded directions interact with the data and the complete model. Low-rank approximation is therefore most effective when matrix-level guarantees, parameter-count constraints, and task-level validation are considered together.

11 Code Availability

For size and formatting reasons, the complete source code is not reproduced within this report. The code used to train the baseline neural network, compute the singular-value decompositions, perform the truncated-SVD compression experiments, conduct the LoRA comparison, calculate the evaluation metrics, and generate the reported results is publicly available on GitHub. The repository can be accessed at:

`View the complete project code on GitHub`

References

[1] Misha Denil, Babak Shakibi, Laurent Dinh, Marc'Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning. In *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013.

[2] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*.

[3] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4 edition, 2013.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

[6] Tara N. Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, and Bhuvana Ramabhadran. Low-rank matrix factorization for Deep Neural Network training with high-dimensional output targets. In *2013 IEEE International Conference on Acoustics, Speech and Signal Processing*, 2013.

[7] Lloyd N. Trefethen and David Bau, III. *Numerical Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, PA, 1997.

[8] Jian Xue, Jinyu Li, and Yifan Gong. Restructuring of deep neural network acoustic models with singular value decomposition. In *INTERSPEECH 2013*, 2013.